



2026

파이썬 기초

posco
포스코인재창조원

교육 목차

시간	학습 내용	학습방법	학습시간
8:00	1. 파이썬 시작하기 - 파이썬 설치, 주피터 노트북 설치, VS code 설치	이론 / 실습	50분
9:00	2. 데이터를 담는 그릇(변수와 자료형) - 변수, 자료형, 표준 입출력		50분
10:00	3. 컴퓨터의 계산기(연산자) - 산술연산자, 비교 연산자, 논리 연산자		50분
11:00	4. 프로그램의 선택(조건문), 반복되는 일처리(반복문) - If ~ else, for, while		50분
12:00	중식		
13:00	5. 데이터의 모듬(자료 구조) - list, dictionary, tuple, set	이론 / 실습	50분
14:00	6. 코드의 조립 및 재사용(함수), 기능 확장(모듈과 패키지) - 함수 정의/활용, 모듈 활용, 내장 및 외부 패키지 활용		50분
15:00	7. 효율적인 설계도(클래스와 객체), 파일 입출력 및 예외 처리 - 클래스의 정의 및 활용, 클래스 상속, 파일 생성하고 읽고 쓰기		50분
16:00	8. gpt 활용 실전! 미니 프로젝트 - 개인 가계부 관리 프로그램, 학생 성적 처리 프로그램, pandas 활용 데이터 분석		50분

01

CHAPTER

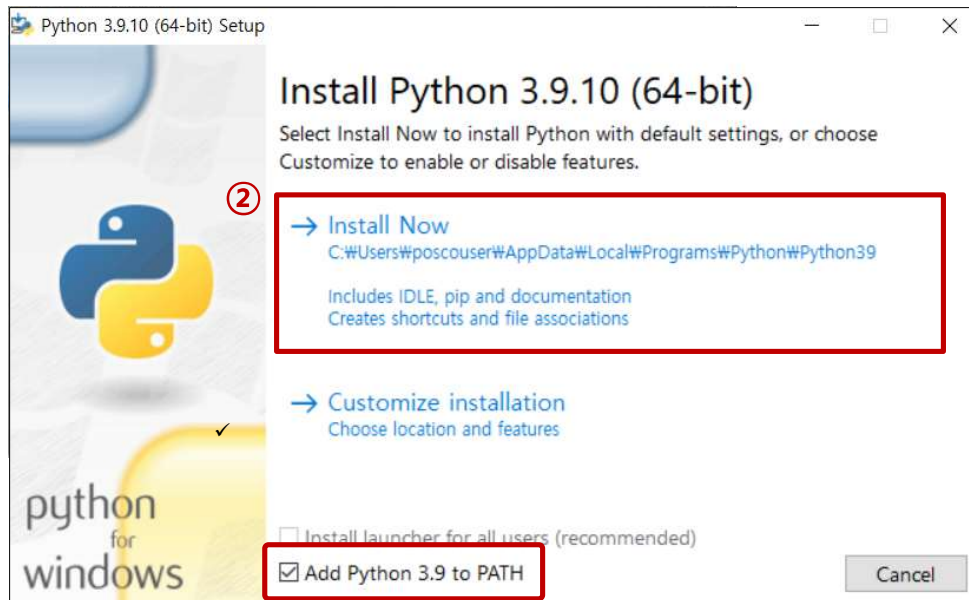
파이썬 시작하기

➤ 파이썬 설치

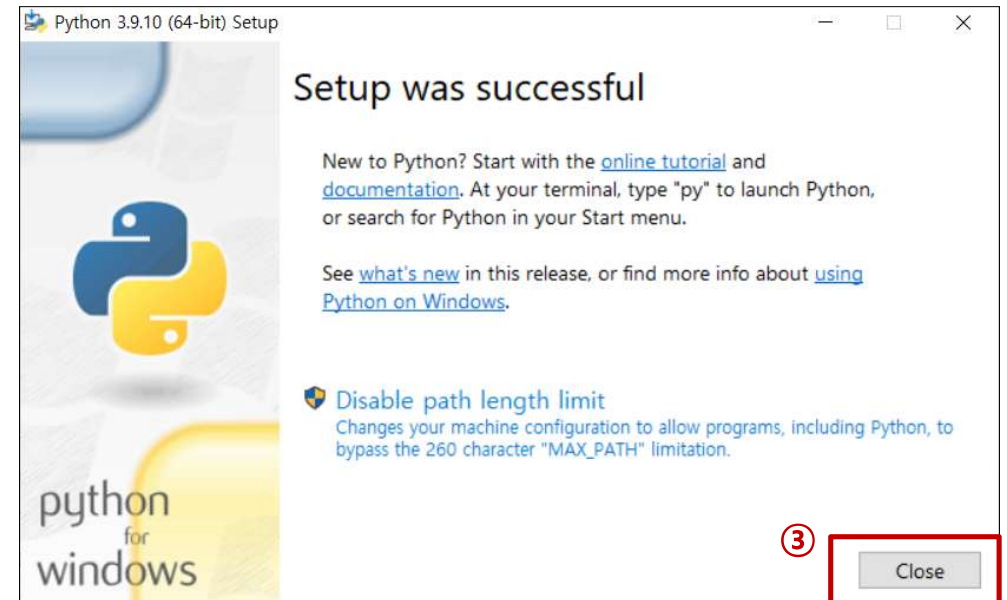
➤ 파이썬 Interpreter : 주피터, Viual Studio Code

- 아래 주소를 크롬 주소창에 붙여넣기 한 후 엔터키로 다운로드 → 다운로드 완료 후 클릭하여 설치

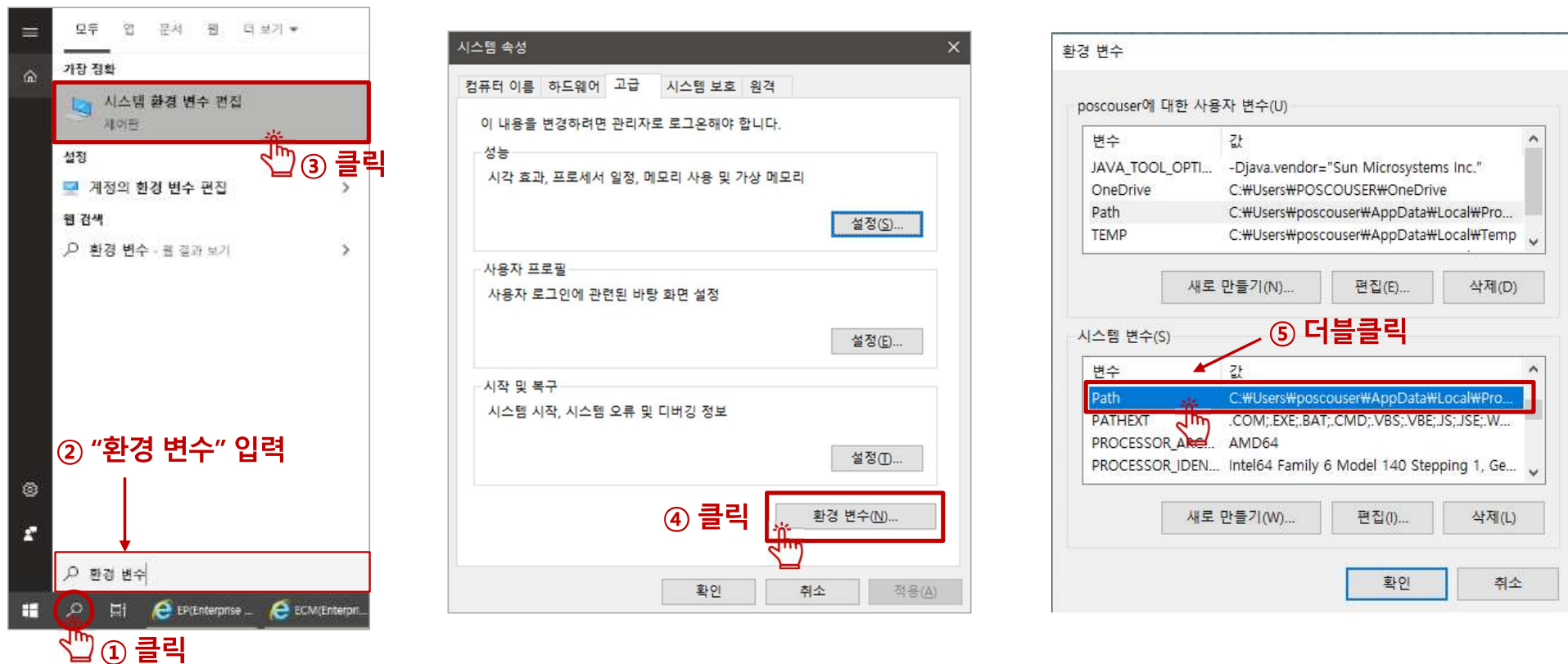
 <https://www.python.org/ftp/python/3.9.10/python-3.9.10-amd64.exe>



① 반드시 체크하세요 !!!



- 모니터 좌측 하단 윈도우키 혹은 돋보기 아이콘 → '환경 변수' 입력 → 엔터키를 눌러 실행



- ✓ '새로 만들기' 버튼을 눌러 아래 경로명을 입력, 확인 버튼을 눌러 저장

The screenshot shows the '환경 변수 편집' (Edit Environment Variables) window. The '시스템 변수' (System variables) tab is active, showing a list of variables. The 'Path' variable is selected. To the right of the list, the '새로 만들기(N)' (New) button is highlighted with a red box and a hand icon, labeled '① 새로 만들기 클릭'. Below the list, a red arrow points from the text '② 경로입력 (복사, 붙여넣기)' to the text input field. Below this, a blue box contains the text '복사, 붙여넣기 할 경로명' (Copy and paste the path name) followed by two lines of the Python path: 'C:\Users\poscouser\AppData\Local\Programs\Python\Python39\' and 'C:\Users\poscouser\AppData\Local\Programs\Python\Python39\Scripts'. At the bottom of the dialog, the '확인' (OK) button is highlighted with a red box and a hand icon, labeled '③ 확인 클릭'.

② 경로입력
(복사, 붙여넣기)

복사, 붙여넣기 할 경로명

C:\Users\poscouser\AppData\Local\Programs\Python\Python39\
C:\Users\poscouser\AppData\Local\Programs\Python\Python39\Scripts

① 새로 만들기 클릭

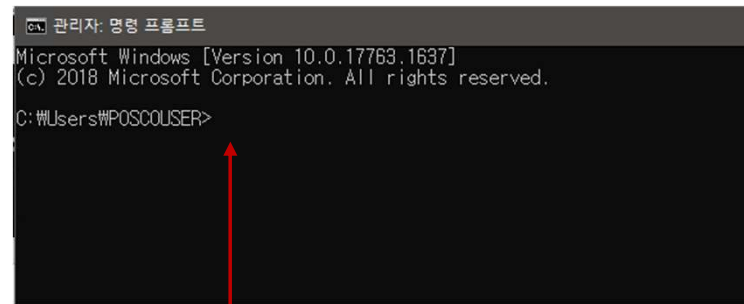
③ 확인 클릭

*pip는 파이썬 패키지 설치를 도와주는 도구입니다.

1) 모니터 좌측 하단 윈도우키 혹은 돋보기 아이콘 → cmd 입력 → 엔터키를 눌러 커맨드창 실행



2) 아래 명령어를 커맨드 창에 복사/붙여넣기 한 다음 엔터키로 실행

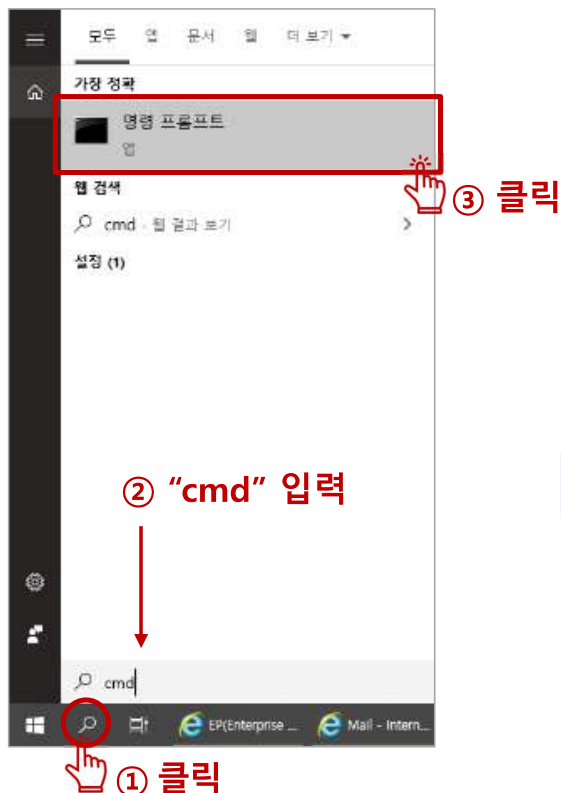


 복사, 붙여넣기 할 경로명

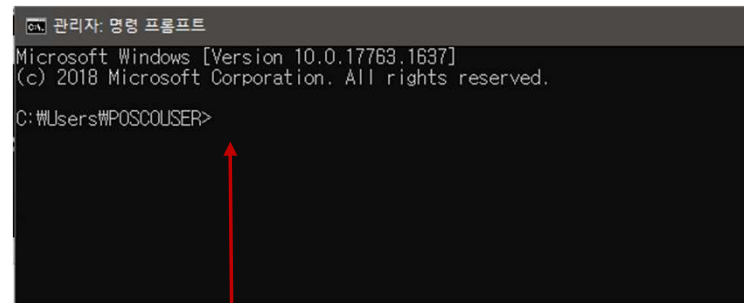
```
python -m pip install --upgrade pip --trusted-host pypi.org --trusted-host files.pythonhosted.org
```

* 커맨드창이 이미 열려있다면 그 창에서 이어서 실행해도 무방합니다.

1) 모니터 좌측 하단 윈도우키 혹은 돋보기 아이콘 → cmd 입력 → 엔터키를 눌러 커맨드창 실행



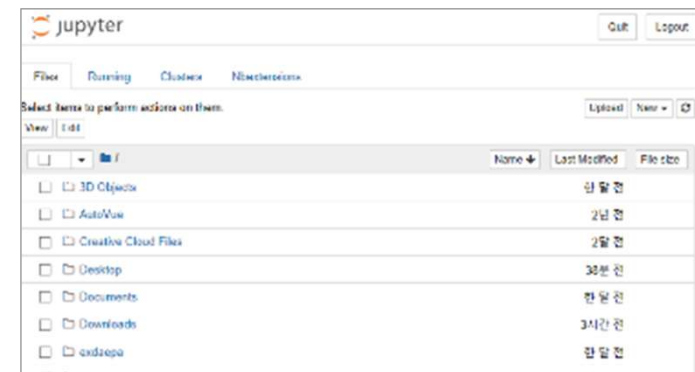
2) 아래 명령어를 커맨드 창에 복사/붙여넣기 한 다음 엔터키로 실행



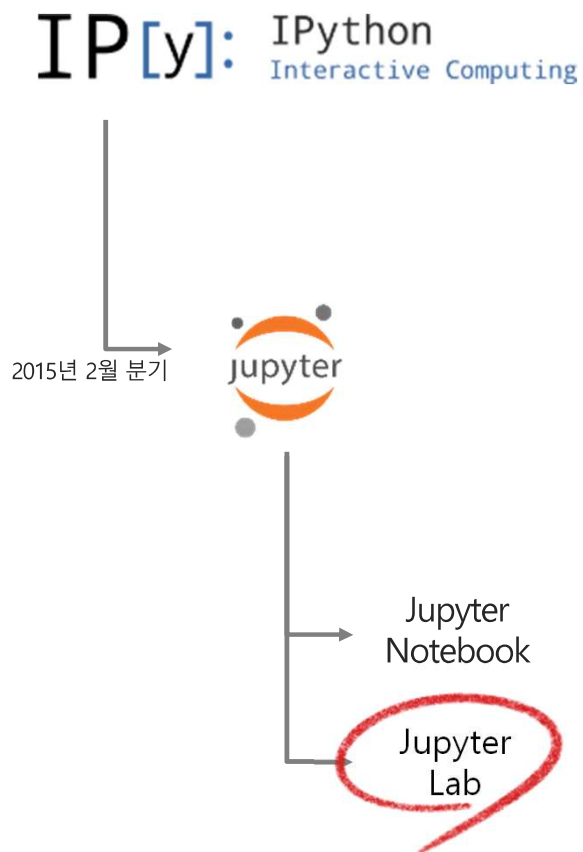
복사, 붙여넣기 할 경로명 | ④

```
pip install jupyter --trusted-host pypi.org --trusted-host files.pythonhosted.org
```

3) 크롬에서 새 창이 열리며 오른쪽 화면이 나타나면 성공



그럼 이제 패키지를 직접 로드해 봅시다. 그 전에 먼저, 주피터 랩에 대해 알아보까요?

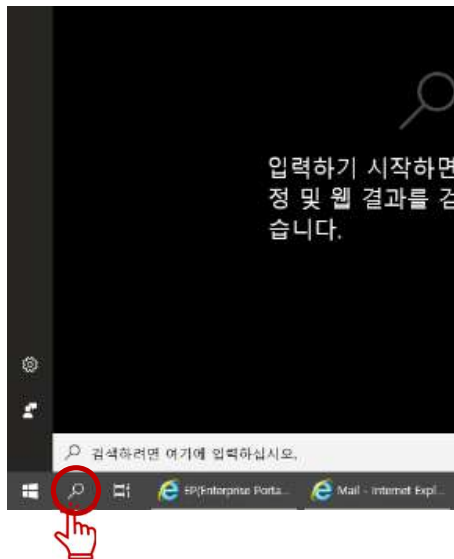


- '01년 파이썬을 활용해 상호작용적인 컴퓨팅을 할 수 있도록 개발된 도구
- 웹브라우저에서 파이썬 코딩을 하고 실행 결과를 바로 확인, 수정할 수 있음
- Ipython은 Interactive Python을 의미

- '15년 2월 Ipython에서 파생된 프로젝트로, 모든 프로그래밍 언어에 걸쳐 인터랙티브 데이터 과학과 사이언티픽 컴퓨팅을 지원하는 것을 목적으로 설립된 비영리 단체
- 주피터의 이름은 주피터가 지원하는 세 개의 핵심 언어인 Julia, Python 그리고 R에서 유래 목성 위성의 발견이 기록된 갈릴레오 갈릴레이의 공책에 대한 존경의 의미도 포함

- Ipython과 마찬가지로 웹브라우저에서 코딩을 하고 실행 결과를 바로 확인, 수정할 수 있는 도구로, Ipython에서 파생되었기 때문에 파이썬으로 프로그래밍한 파일은 [파일명.ipynb] 형태로 저장됨(ipynb : Ipython notebook)

- 주피터 노트북의 차세대 버전으로 데이터 사이언스와 머신러닝에 특화된 인터페이스와 모듈, 그리고 다양한 확장기능 제공



모니터 좌측 하단
돋보기 모양 아이콘 클릭

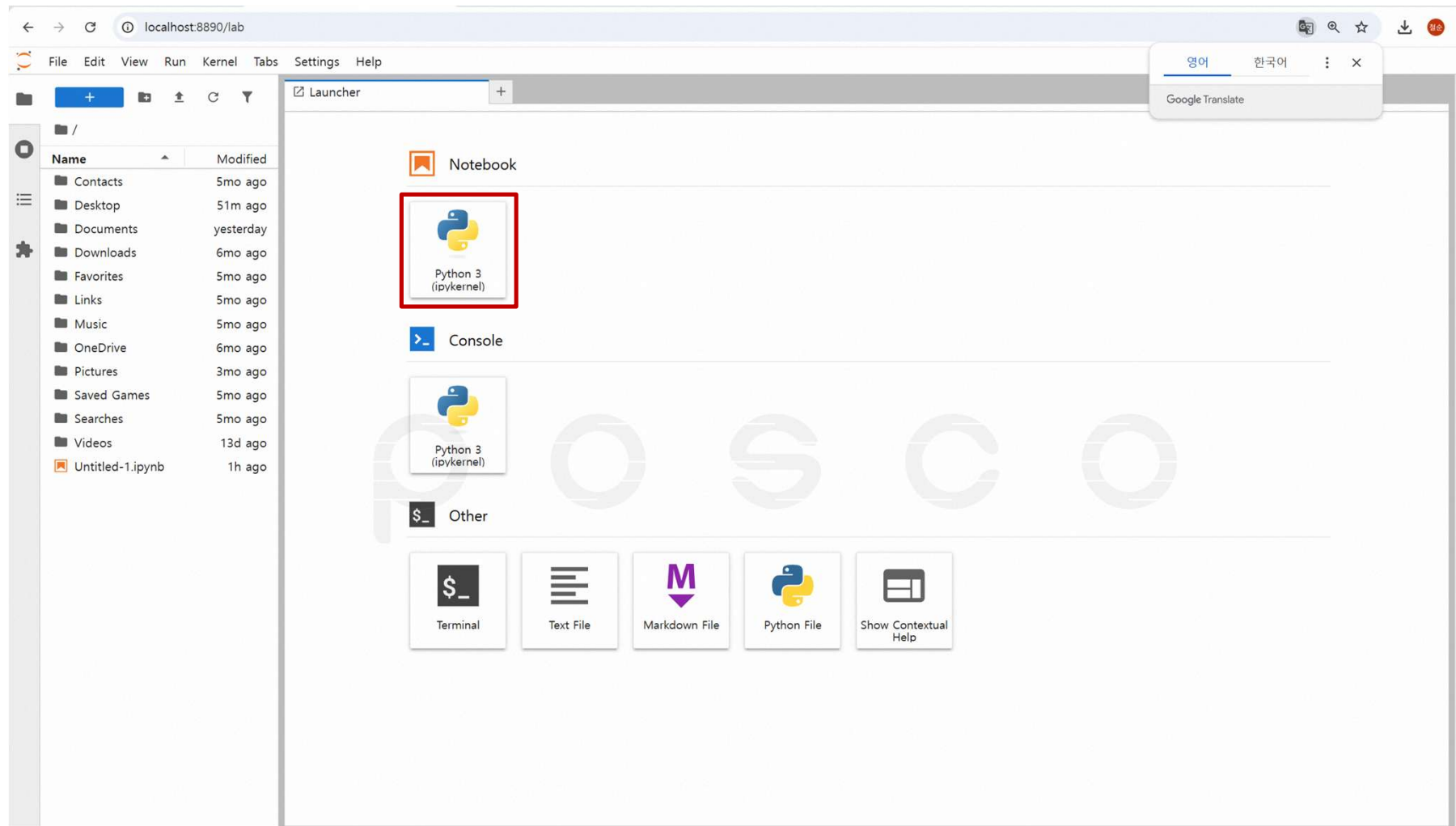


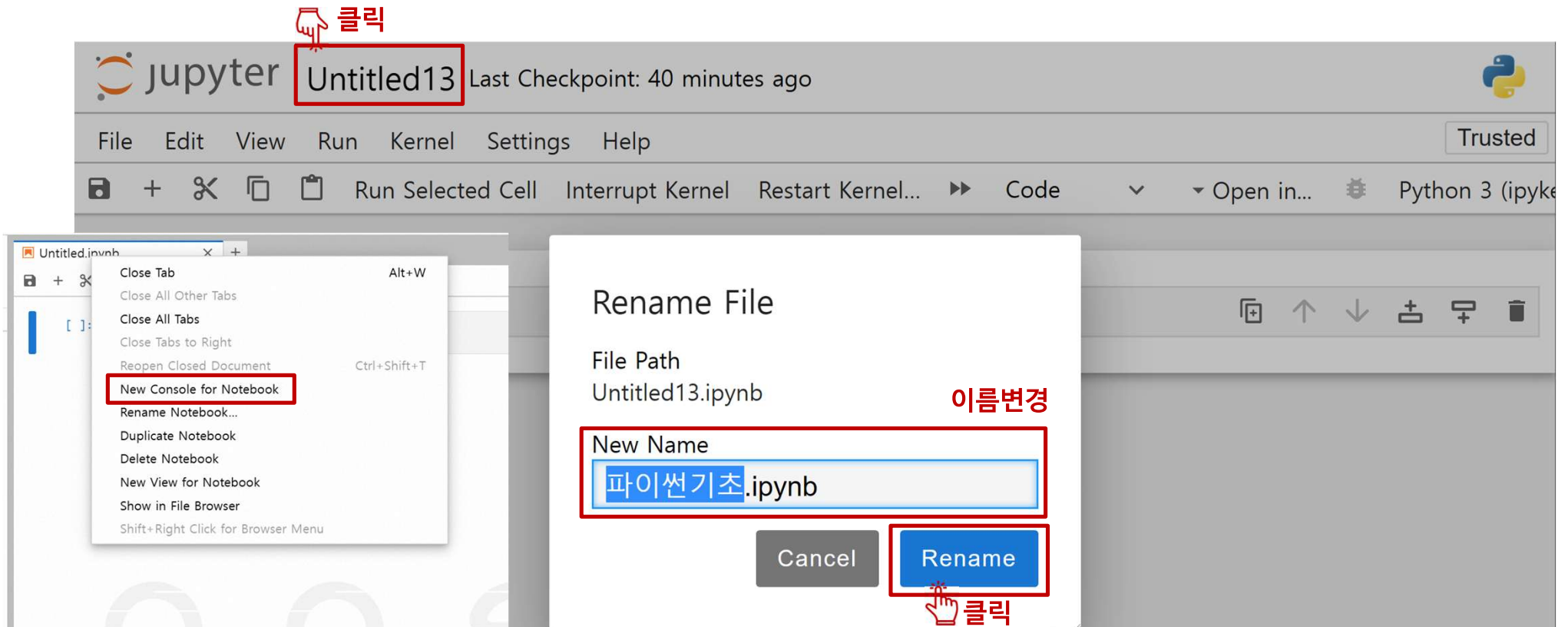
「cmd」 입력한 후 엔터
→ 커맨드창 실행



```
Microsoft Windows [Version 10.0.26100.7462]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\poscouser>jupyter lab  
'jupyter'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는  
배치 파일이 아닙니다.  
  
C:\Users\poscouser>jupyter lab
```

「jupyter lab」 입력한 후 엔터 → 주피터랩 실행





'파이썬 기초' 로 입력하고 Rename 버튼을 눌러 저장

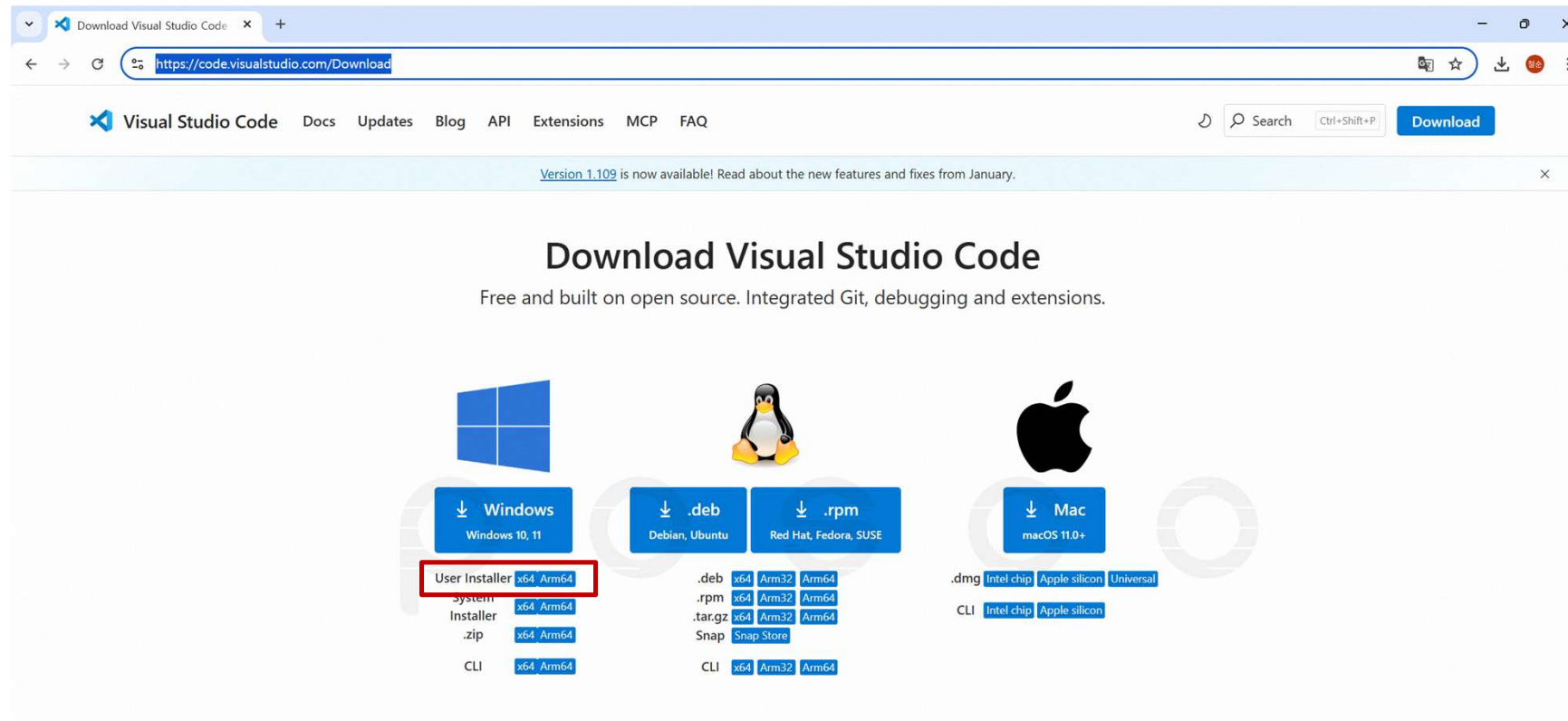
현재 선택한 셀의 코드를 실행(Ctrl+Enter) 실행결과는 선택한 셀 하단 아웃풋 창에 출력

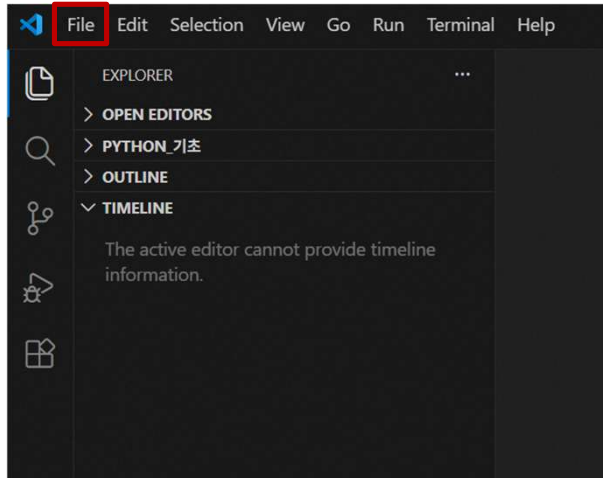
The image shows a Jupyter Notebook interface. At the top, the Jupyter logo and '파이썬기초' (Python Basics) are visible, along with 'Last Checkpoint: 45 minutes ago'. The menu bar includes File, Edit, View, Run, Kernel, Settings, and Help. The 'Run' menu is open, showing options like 'Run Selected Cell', 'Interrupt Kernel', and 'Restart Kernel...'. The 'Run Selected Cell' option is highlighted with a red box. Below the menu bar, there are icons for file operations and a 'Code' dropdown menu. The main area contains two code cells. The first cell, labeled '셀(cell)' on the left, contains the code: `[1]: import numpy` and `print(numpy.__version__)`. Below this cell, the output '1.24.2' is displayed, labeled '아웃풋' on the left. The second cell contains the code: `[2]: print("안녕하세요, 파이썬이 이런거군요!")` and the output '안녕하세요, 파이썬이 이런거군요!'. A third cell is partially visible at the bottom, showing the prompt `[ ]:`.

셀은 여러 개 만들 수 있으며, 셀 단위로 코드 실행이 가능

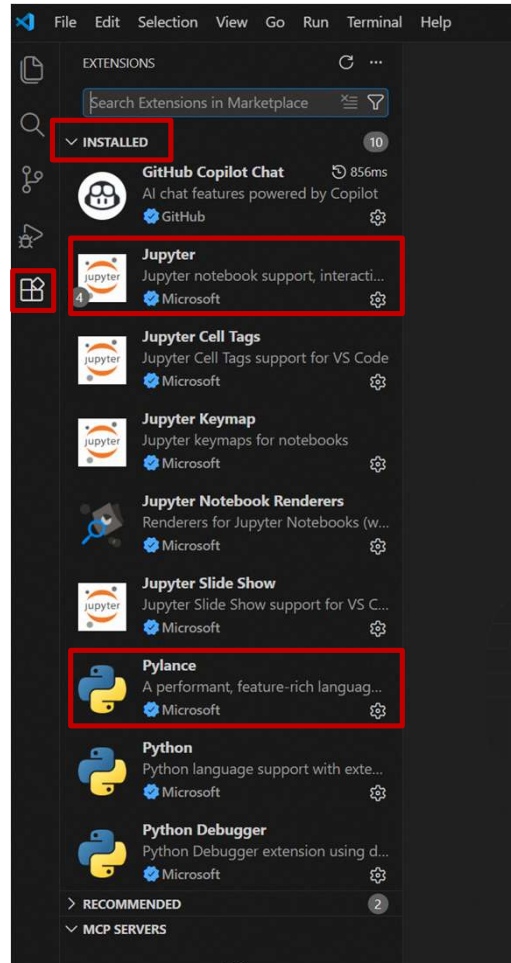
- 아래 주소를 크롬 주소창에 붙여넣기 한 후 엔터키로 다운로드 → 다운로드 완료 후 클릭하여 설치

 <https://code.visualstudio.com/Download>

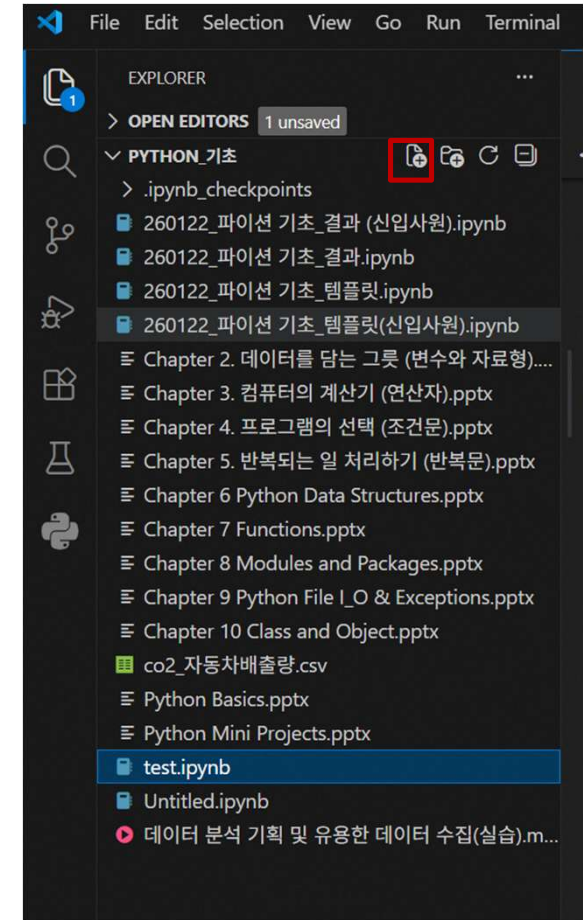




「파일 홀더」 지정

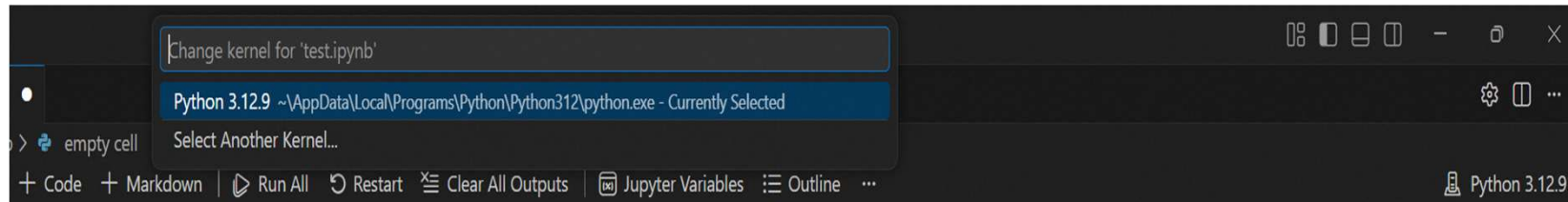


「Jupyter와 python」 extension 설치 확인

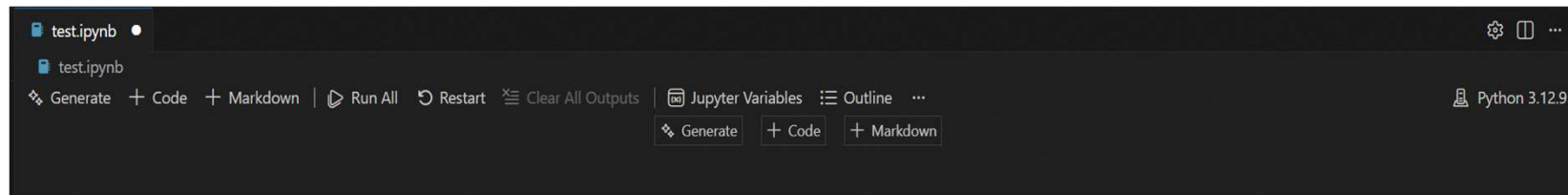


「파일」생성하기

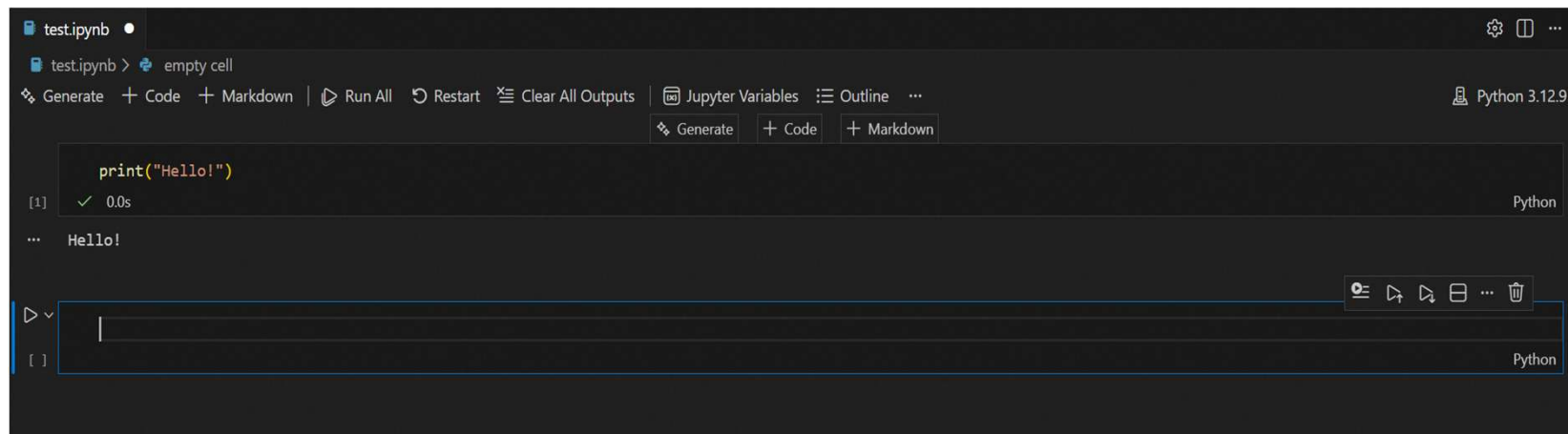
- 「Kernel」을 Python으로 지정



- Code 셀 추가



- Code 입력하고 실행하기



- 1 파일 만들기 :** jupyter에서 python을 누르고, hello.pynb라는 이름으로 저장합니다.
(“.pynb”는 주피터노트북 파이썬 파일의 확장자입니다.)

2 코드 입력

```
print("Hello!") # 안녕하세요
```

- 3 실행 :** 상단의 재생 버튼(▶)을 누르거나, enter 또는 shift+ enter를 칩니다.

- 4 결과 :** 터미널 창에 Hello World!라는 문자가 나타나면 성공입니다.
print()는 괄호 안의 내용을 화면에 보여주는 출력 함수입니다.

```
[1]: print("hello") # 안녕하세요
```

hello

* 컴퓨터는 무시하고 사람만 볼 수 있는 메모를 주석이라고 합니다.

종류	기호	설명
한 줄 주석	#	# 뒤에 오는 모든 내용은 무시됩니다.
여러 줄 주석	""" 또는 '''	따옴표 3개로 감싸서 여러 줄을 작성합니다.

➤ 코딩을 처음 할 때 가장 많이 만나는 붉은색 에러 메시지 3대장입니다. 당황하지 말고 에러의 이름을 확인하세요.

에러 이름	원인	해결 방법
SyntaxError	문법 오류	오타가 있거나, 괄호()나 따옴표""의 짝이 맞지 않는지 확인하세요.
IndentationError	들여쓰기 오류	파이썬은 들여쓰기(띄어쓰기)가 매우 중요합니다. 줄 간격이 맞는지 확인하세요.
NameError	이름 오류	선언하지 않은 변수명을 사용했거나, 변수 이름의 철자가 틀렸을 때 발생합니다.

02

CHAPTER

데이터를 담는 그릇 (변수와 자료형)

- 변수
- 자료형
- 표준 입출력

- 값이 변하지 않는 숫자, 문자, 문자열과 같은 고정된 값
- 숫자는 상수
- 홑따옴표(') 혹은 쌍따옴표(")로 표시된 문자열은 상수

100을 화면에 출력

```
print(100)
```

100.123을 화면에 출력

```
print(100.123)
```

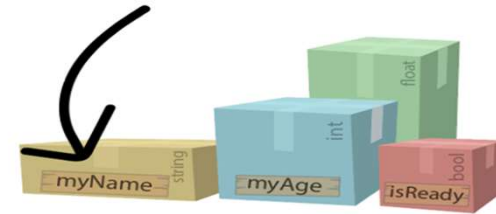
'안녕하세요'를 화면에 출력

```
print('안녕하세요') # 홑따옴표, 쌍따옴표 모두 출력 값은 동일
```

```
print("안녕하세요")
```

➤ 변수는 데이터를 저장하는 상자(메모리 공간)입니다.

- 선언 : 변수명 = 값
- 의미 : 수학의 '같다'가 아니라, "오른쪽의 값을 왼쪽 상자에 담는다(할당)"는 의미입니다.



```
name = "파이썬"
age = 20
print(name)  # '파이썬'이라는 글자가 들어있는 상자를 열어 출력
```

• 변수 작성 규칙

규칙	설명 및 예시
사용 가능 문자	영문자, 숫자, 밑줄(_)만 사용 가능합니다.
숫자 시작 불가	숫자로 시작할 수 없습니다. (예: 1st 불가, st1 가능)
예약어 사용 불가	파이썬 문법 키워드(if, for, while 등)는 사용할 수 없습니다.
관례 (Snake Case)	보통 소문자와 밑줄을 섞어 씁니다. (예: my_score)

- 예약어는 파이썬이 정한 의미로만 쓰이는 특별한 단어
- 예약어는 변수 이름이나 메소드 이름 등 다른 식별자 이름으로 사용 불가

SyntaxError: can't assign to keyword # 예약어를 변수로 사용하면 출력되는 메시지

- 예약어는 대소문자 구별. True, False, None을 제외하고는 모두 소문자
- 예약어 종류

and	continue	except	global	lambda	pass	while
as	def	False	if	None	raise	with
assert	del	finally	import	nonlocal	return	yield
break	elif	for	in	not	True	
class	else	from	is	or	try	

- 모든 프로그래밍 언어는 '자료형' 이라는 개념을 가지고 있습니다.
- Python은 **동적 타이핑 언어**로, 변수 선언 시 자료형을 미리 지정하지 않아도 됨(변수에 자료 할당 시 자동 결정)

구분	자료형	이름	특징	예시
기본 자료형	int	정수형	소수점이 없는 숫자	10, -5, 0
	float	실수형	소수점을 포함한 숫자	3.14, -0.1, 2.0
	str	문자열	텍스트 데이터 (따옴표 사용)	"Hello", 'Python'
	bool	논리형	참과 거짓을 판단	True, False
자료 구조형	list	리스트	여러 값을 순서대로 저장, 수정 가능	[1, 2, 3], ['a', 'b']
	tuple	튜플	여러 값을 순서대로 저장, 수정 불가	(1, 2, 3), ('x', 'y')
	set	집합	중복을 허용하지 않음, 순서 없음	{1, 2, 3}, {1, 1, 2} → {1, 2}
	dict	딕셔너리	키(Key)와 값(Value)의 쌍으로 구성	{"name": "Tom", "age": 20}
-	None Type	None	값이 존재하지 않음을 명시적으로 표현	None

- 숫자형(Number)이란 숫자 형태로 이루어진 자료형
- 정수형(Integer) : 음수, 0, 양수로 구성된 숫자
2진수(0b), 8진수(0o), 16진수(0x) 등 다양한 형식 지원(0xFF : 255의 16진수)
- 실수형(Floating Point) : -0.37, -33.0, 37.33에서와 같이 소수점을 가진 숫자
- 'e'를 사용해 아주 크거나 작은 수를 간결하게 표현(1.2e2 : 0.012)

```
a = 10      # 정수
b = 3.14    # 실수
print(a + b) # 13.14 (정수와 실수를 더하면 실수가 됩니다)
```

TIP!

변수의 자료형이 궁금할 땐 `type()` 함수를 써보세요. `print(type(a))` -> `<class int>` 라고 나옵니다.

인덱싱	변수명[번호]
슬라이싱	변수명[시작:끝]

- 문자열은 각 글자마다 번호(인덱스)가 매겨져 있습니다. **0부터 시작**한다는 점을 주의하세요!
- 슬라이싱은 **끝 번호 앞**까지만 가져옵니다.

```
s = "Python"  # P(0) y(1) t(2) h(3) o(4) n(5)
print(s[0])   # 'P' (첫 번째 글자)
print(s[-1])  # 'n' (뒤에서 첫 번째 글자)
print(s[0:2]) # 'Py' (0 번부터 2 번 직전인 1 번까지)
print(s[2:])  # 'thon' (2 번부터 끝까지)
```

- 문자열 중간에 변수 값을 끼워 넣을 때 가장 편리한 방법입니다.
- 문자열 앞에 f를 붙이고 중괄호 {} 안에 변수를 넣습니다.
- **표현식 계산 가능** : 중괄호 안에서 산술 연산이나 함수 호출이 가능합니다.

```
name = "철수"
score = 90
print(f"{name}의 점수는 {score}점입니다.")

print(f"2 곱하기 3은 {2*3}입니다.")
```

- **소수점 제어 및 천 단위 콤마** : 숫자를 표현할 때 아주 유용합니다.
소수점 : {변수:.2f} (소수점 둘째 자리까지)
콤마 : {변수:,} (천 단위 구분 기호)
- **정렬과 공백**
우측 정렬 : f"{name:>10}" (10칸 확보 후 오른쪽 정렬)
좌측 정렬 : f"{name:<10}" (10칸 확보 후 왼쪽 정렬)
가운데 정렬 : f"{name:^10}" (10칸 확보 후 가운데 정렬)

AI에게 질문을 던지는 '프롬프트 템플릿' 작성 시,
사용자의 입력에 따라 내용을 바꾸는 핵심 기술이 바로 **f-string**입니다.

1. AI에게 부여할 역할(Persona) 설정

```
role = "전문 번역가"
```

2. 사용자가 입력한 문장

```
user_text = "안녕하세요, 오늘 날씨가 참 좋네요."
```

3. f-string을 이용해 프롬프트(명령어) 자동 완성

```
prompt = f"당신은 {role}입니다. 다음 문장을 영어로 번역해주세요: '{user_text}'"
```

```
print(prompt)
```

결과: 당신은 전문 번역가입니다. 다음 문장을 영어로 번역해주세요: '안녕하세요, 오늘 날씨가 참 좋네요.'

- 문자열 자료형은 자체적으로 데이터를 처리할 수 있는 다양한 기능(함수)을 가지고 있습니다. 이를 메서드라고 부르며 변수명.메서드이름() 형태로 사용합니다.

메서드	설명	사용 예시	결과
count	특정 문자의 개수를 셉니다.	"hobby".count('b')	2
find	특정 문자가 처음 나온 위치(인덱스)를 찾습니다. (없으면 -1)	"Python".find('y')	1
index	find와 같지만, 찾는 문자가 없으면 에러가 납니다.	"Python".index('t')	2
join	문자열 사이에 특정 문자를 삽입합니다.	",".join("abcd")	"a,b,c,d"
upper	소문자를 대문자로 바꿉니다.	"hi".upper()	"HI"
lower	대문자를 소문자로 바꿉니다.	"HI".lower()	"hi"
strip	양쪽 공백을 지웁니다. (lstrip: 왼쪽, rstrip: 오른쪽)	" hi ".strip()	"hi"
replace	특정 문자열을 다른 문자열로 바꿉니다.	"Life is short".replace("short", "long")	"Life is long"
split	문자열을 특정 구분자를 기준으로 나눕니다. (리스트 반환)	"a:b:c".split(":")	['a', 'b', 'c']

활용 예시

```
text = " Python Programming "
```

1. 양쪽 공백 제거 후 모두 대문자로 변환

```
clean_text = text.strip().upper()
```

```
print(clean_text)    # "PYTHON PROGRAMMING"
```

2. 문자열 바꾸기

```
new_text = clean_text.replace("PYTHON", "JAVA")
```

```
print(new_text)      # "JAVA PROGRAMMING"
```

- 문자열 안에 줄바꿈이나 따옴표 기호 자체를 넣고 싶을 때 역슬래시(\)를 사용합니다.

코드	설명	예시
\n	줄바꿈 (Enter)	print("안녕\n하세요")
\t	탭 (Tab)	print("이름\t나이")
\", \'	따옴표 출력	print("그가 \"안녕\"이라고 말했다")

➤ 문자열 : 연산(더하기, 곱하기)

- + (더하기): 두 문자열을 연결합니다.
- * (곱하기): 문자열을 횟수만큼 반복합니다.

```
print("안녕" + "하세요")    # "안녕하세요"  
print("ㅋ" * 5)             # "ㅋㅋㅋㅋㅋ"
```

- Boolean(불리언)은 데이터 타입 중 하나로, 논리적인 판단을 할 때 사용되는 **True**(참, 첫 글자가 대문자)와 **False**(거짓) 두 가지 값 만을 가집니다.
- 프로그래밍에서 주로 **비교 연산자**나 **논리 연산자**의 결과값으로 생성되어 흐름을 제어하는 데 핵심적인 역할 수행.

```
is_python_fun = True
is_math_hard = False
print(10 > 5) # 출력: True
print(10 == 5) # 출력: False
```

➤ 불리언의 진리값 (Truthiness)

파이썬에서는 bool() 함수를 사용하여 일반 데이터가 True인지 False인지 확인할 수 있습니다.
거의 모든 값은 True로 간주되지만, **비어 있거나 0인 값**은 False가 됩니다.

➤ False로 평가되는 것들

- 숫자 : 0, 0.0
- 빈 컨테이너 : "" (빈 문자열), [] (빈 리스트), {} (빈 튜플), {} (빈 딕셔너리)
- 기타 : None, False 그 자체

➤ True로 평가되는 것들

- 값이 들어있는 문자열, 리스트, 딕셔너리 ("abc", [1, 2])
- 0이 아닌 숫자 (1, -5, 3.14)

- 자료형을 다른 형태로 강제로 바꾸는 것입니다.

함수	설명	예시
int(값)	정수로 변환	int("10") → 10
float(값)	실수로 변환	float(5) → 5.0
str(값)	문자열로 변환	str(100) → "100"

- print() 옵션

end 출력 후 줄바꿈 대신 다른 문자를 넣고 싶을 때

```
print("안녕", end="!")  
print("하세요")  
# 결과: 안녕!하세요 (줄바꿈 없이 이어짐)
```

sep 여러 값을 출력할 때 사이사이에 넣을 문자 (Separator)

```
print("010", "1234", "5678", sep="-")  
# 결과: 010-1234-5678
```

엑셀 파일 등을 불러오면 숫자가 문자로 인식되는 경우가 많습니다.
이때 int()나 float()를 사용해 올바른 자료형으로 바꿔주는 작업이 필수적입니다.

엑셀이나 웹에서 긁어온 데이터는 보통 문자열 형태입니다.

```
raw_price = "1,500"
```

1. 쉼표(,)를 제거합니다. "1,500" -> "1500"

```
clean_str = raw_price.replace(",", "")
```

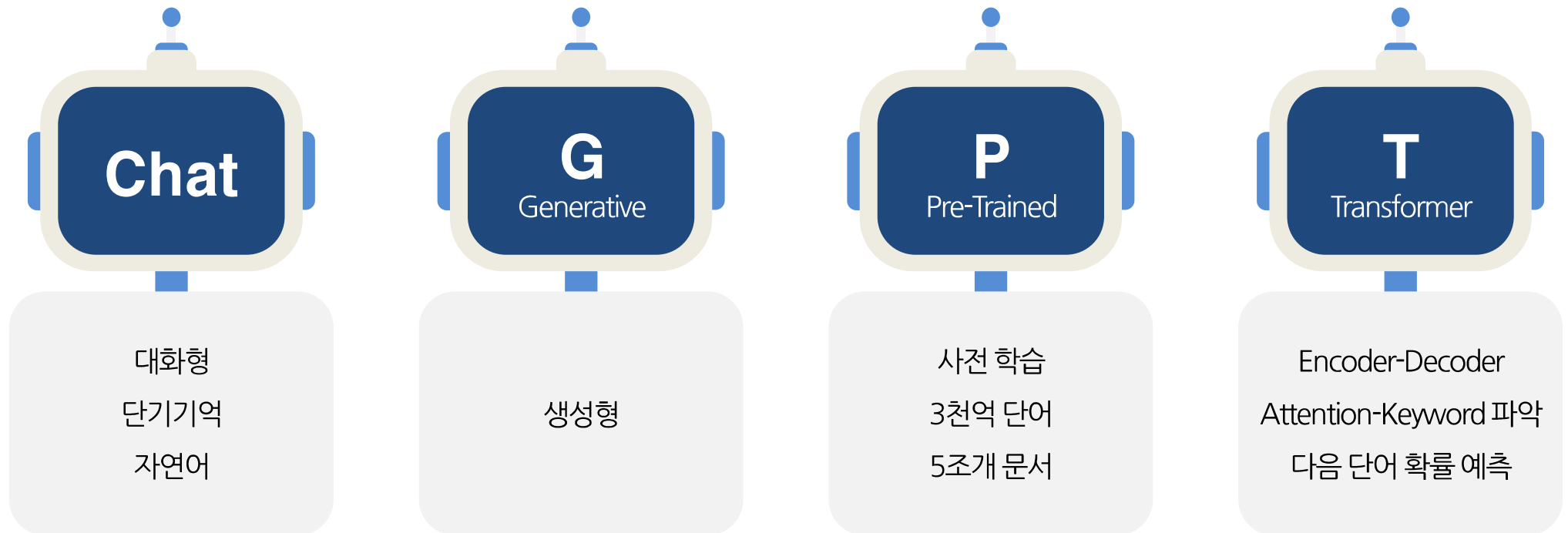
2. 계산을 위해 정수(int)로 변환합니다.

```
price = int(clean_str)
```

3. 이제 덧셈 계산이 가능합니다!

```
print(f"원래 가격: {price}원")
```

```
print(f"배송비 포함: {price + 3000}원")
```



☑ 특별한 규칙이 있는 것은 아니지만, 체계적 질문을 통해 양질의 답변을 얻을 수 있음

역할

- 역할, 대상

**상황**

- 맥락, 사전 지식, 참조

**지시**

- 조건, 가이드라인

**출력 형태**

- 예시, 포맷

연습

다음 정보를 변수로 저장한 후 **f-string**을 사용하여 출력하는 프로그램을 작성하세요.

변수	값
이름	홍길동
부서	데이터분석팀
교육시간	40

출력 결과 예시 홍길동님은 데이터분석팀 소속이며 Python 교육을 40시간 수강했습니다.

Prompt

[역할] 너는 파이썬 개발 전문가야

[상황] 파이썬 f-string을 사용해서 출력하는 코드를 생성하고자 해

[지시] 홍길동이는 데이터분석팀이고, 파이썬 교육을 40시간 수강했어를 출력해줘

[출력] 이름, 부서, 교육시간 순서를 문장을 출력해줘

```
## 프롬프트 연습 (31페이지)
```

```
# 변수 설정
```

```
name = "홍길동"
```

```
department = "데이터분석팀"
```

```
education_hours = 40
```

```
# 이름, 부서, 교육시간 순서로 f-string 출력
```

```
print(f"{name}이는 {department}이고, 파이썬 교육을 {education_hours}시간 수강했어")
```

✓ 0.0s

홍길동이는 데이터분석팀이고, 파이썬 교육을 40시간 수강했어

Q 문제

여러분은 PC방의 알바생입니다.

손님이 이용한 시간과 분식을 입력받아 최종 결제 금액을 알려 주는 프로그램을 작성하세요.

[가격표] - 1시간 이용료 : 1,000원

- 음료수 가격 : 2,000원

[입력 예시] - 3시간 : 3

- 음료수 개수 : 2

TIP! input() 함수 : 사용자가 입력한 값을 받아옵니다. 입력받은 모든 값은 문자열입니다. 계산하려면 형 변환이 필요합니다.

Prompt

[역할] 너는 파이썬 개발 전문가야

[상황] PC방 아르바이트생을 위한 자동 결제 금액 계산 프로그램을 작성하려고 해.

[지시] 아래 순서대로 진행되는 파이썬 코드를 생성해줘

1. 사용자로부터 이용 시간과 음료수 개수를 각각 정수형으로 입력받을 것.

- 가격표 정보 : 1시간 이용료는 1,000원이고, 음료수 가격은 2,000원

2. 총 결제 금액을 계산하는 수식을 만들고, f-string을 사용하여 결과를 출력할 것.

[출력] 출력 문구 예시: "이용시간은 3시간이고, 음료는 2개, 총 결제 금액은 7,000원입니다."

파이썬 코드 예시

```
# [지시 1] 사용자로부터 이용 시간과 음료수 개수를 정수형(int)으로 입력받기
# input() 함수는 입력받은 데이터를 문자열(str)로 취급하므로 산술 연산을 위해 int()로 변환합니다.
hours = int(input("이용 시간을 입력하세요 (시간 단위): "))
drinks = int(input("구매한 음료수 개수를 입력하세요: "))

# 가격 정보 상수 설정 (유지보수가 쉽도록 변수로 지정)
PRICE_PER_HOUR = 1000 PRICE_PER_DRINK = 2000

# [지시 2] 총 결제 금액 계산 수식 및 f-string 출력
# 총 금액 = (시간 * 1000) + (음료수 * 2000)
total_payment = (hours * PRICE_PER_HOUR) + (drinks * PRICE_PER_DRINK)

# 결과 출력 (천 단위 콤마 추가로 가독성 높임)
print(f"이용시간은 {hours}시간이고, 음료는 {drinks}개, 총 결제 금액은 {total_payment:,}원입니다.")
```

03

CHAPTER

컴퓨터의 계산기 (연산자)

➤ 산술연산자

➤ 비교연산자

➤ 논리연산자

➤ 사칙연산과 특수 연산

연산자	의미	설명 및 예시
+	더하기	$3 + 2 \rightarrow 5$
-	빼기	$3 - 2 \rightarrow 1$
*	곱하기	$3 * 2 \rightarrow 6$
/	나누기	$4 / 2 \rightarrow 2.0$ (항상 실수 결과)
//	몫	$10 // 3 \rightarrow 3$ (나눗셈의 정수 부분)
%	나머지	$10 \% 3 \rightarrow 1$ (짝수/홀수 판별에 유용)
**	거듭제곱	$2 ** 3 \rightarrow 8$ (2^3)

```
print(10 // 3) # 3 (10을 3으로 나눈 몫)
print(10 % 3) # 1 (10을 3으로 나눈 나머지)
```

➤ 비교 연산자

두 값을 비교하여 결과가 True 또는 False로 나옵니다.

연산자	의미
==	같다 (등호가 2개임을 주의!)
!=	다르다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다

```
x = 10
y = 20
print(x == y) # False
print(x != y) # True
```

논리 연산자

여러 조건을 묶어서 판단할 때 씁니다.

연산자	의미	설명
and	그리고	둘 다 참이어야 True
or	또는	둘 중 하나라도 참이면 True
not	반대	참을 거짓으로, 거짓을 참으로 뒤집음

```
age = 25
has_ticket = True
# 나이가 20 이상이고(and) 티켓이 있어야 입장 가능
if age >= 20 and has_ticket:
    print("입장 가능")
```

사용자의 질문

```
user_input = "오늘 놀이공원 영업하나요?"
```

규칙: 질문에 '영업'이나 '시간'이라는 단어가 있고(and) '놀이공원'이 포함되어 있으면, "놀이공원 영업시간은 09:00 ~ 22:00 입니다."안내하고 아니면 "죄송합니다"

```
if ("영업" in user_input or "시간" in user_input) and "놀이공원" in user_input:
```

```
    print("챗봇: 놀이공원 영업시간은 09:00 ~ 22:00 입니다.")
```

```
else:
```

```
    print("챗봇: 죄송합니다. 무슨 말씀인지 모르겠어요.")
```

➤ 복합 대입 연산자

변수의 값을 자신과 연산하여 다시 저장할 때 줄여 쓰는 문법입니다.

연산자	의미	설명
<code>a += 1</code>	<code>a = a + 1</code>	a를 1 증가시킴
<code>a -= 1</code>	<code>a = a - 1</code>	a를 1 감소시킴
<code>a *= 2</code>	<code>a = a * 2</code>	a를 2배로 만들
<code>a /= 2</code>	<code>a = a / 2</code>	a를 2로 나눔

Q 문제

사용자로부터 '초(second)' 단위의 시간을 입력받아, 몇 시간, 몇 분, 몇 초인지 변환하여 알려주는 프로그램을 작성하세요.

[예시] 3700초를 입력하면 → "1시간 1분 40초"가 출력되어야 합니다.

[힌트] 1분 = 60초, 1시간 = 60분 (3600초)

- ① 시간을 구하려면 3600으로 나눈 몫을 구합니다.
- ② 분 계산 (남은 초를 60으로 나눈 몫)
- ③ 초 계산 (남은 초를 60으로 나눈 나머지)

Prompt

... [역할] 너는 파이썬 개발 전문가야

[상황] 사용자로부터 입력받은 '초(second)'를 시, 분, 초 단위로 변환하는 프로그램을 만들려고 해.

[지시] input() 함수를 사용하여 사용자로부터 '초' 단위의 정수를 입력받을 것.

다음의 산술 연산을 활용하여 시, 분, 초를 계산할 것: 1시간 = 3600초 / 1분 = 60초

몫 연산자(//)와 나머지 연산자(%)를 반드시 사용할 것.

[출력] 결과는 f-string을 사용하여 출력할 것.

출력 문구 예시: "3700초는 1시간 1분 40초입니다."

파이썬 코드 예시

```
# 1. 사용자로부터 '초' 단위의 정수 입력받기
total_seconds = int(input("변환할 초를 입력하세요: "))

# 2. 산술 연산을 활용하여 시, 분, 초 계산 # 전체 초를 3600으로 나눈 몫은 '시'가 됩니다.
hour = total_seconds // 3600

# 전체 초를 3600으로 나눈 나머지를 다시 60으로 나눈 몫은 '분'이 됩니다.
minute = (total_seconds % 3600) // 60

# 마지막으로 60으로 나누고 남은 나머지는 '초'가 됩니다.
second = total_seconds % 60

# 3. f-string을 사용하여 결과 출력
print(f"{total_seconds}초는 {hour}시간 {minute}분 {second}초입니다.")
```

04

CHAPTER

프로그램의 선택 (조건문)

➤ If ~ else

➤ 중첩 if문

➤ 기본 구조

파이썬의 if문은 **if**, **elif**, **else** 세 가지 키워드를 사용합니다.

가장 중요한 점은 조건문 뒤에 콜론(:)을 붙이고, 실행할 코드는 반드시 들여쓰기(indentation)를 해야 한다는 것입니다.

```
if 조건식:
    # 조건이 참일 때 실행할 코드
elif 조건식:
    # 이전 조건이 거짓이고, 현재 조건이 참일 때 실행할 코드
else:
    # 위의 모든 조건이 거짓일 때 실행할 코드
```

➤ 비교 연산자와 논리 연산자 활용

조건식에는 다양한 연산자를 조합하여 정교한 조건을 만들 수 있습니다.

- **비교**: == (같다), != (다르다), >, <, >=, <=
- **논리**: and, or, not

```
age = 25
has_ticket = True
if age >= 20 and has_ticket:
    print("입장이 가능합니다.")
```

➤ 양자 택일 : if ~ else 문

조건이 참이면 if 블록을, 거짓이면 else 블록을 실행합니다.

```
if money >= 3000:
    print("택시를 탄다")
else:
    print("걸어간다") # 조건이 거짓일 때 실행
```

➤ 다중 선택 : if ~ elif ~ else

조건이 여러 개일 때 사용합니다. elif(else if의 줄임말)는 개수 제한 없이 쓸 수 있습니다.

위에서부터 차례대로 검사하며, 하나라도 참이면 나머지는 건너뜁니다.

```
score = 85
if score >= 90:
    print("A학점")
elif score >= 80:
    print("B학점") # 여기서 참이 되므로 실행하고 if문 종료
elif score >= 70:
    print("C학점")
else:
    print("F학점")
```

➤ 중첩 조건문

if문 안에 또 다른 if문을 넣을 수 있습니다.

```
age = 20
card = True
if age >= 19:
    if card:
        print("결제 진행")
    else:
        print("카드가 필요합니다")
else:
    print("미성년자는 구매 불가")
```

05

CHAPTER

반복되는 일 처리하기 (반복문)

➤ for 문

➤ While문

- 특정 횟수만큼 반복하거나, 리스트 같은 데이터 묶음을 하나씩 꺼낼 때 주로 사용합니다.
- range() 함수와 함께 자주 쓰입니다.
- range() 함수

코드	생성되는 숫자	횟수
range(5)	0, 1, 2, 3, 4	5회
range(1, 6)	1, 2, 3, 4, 5	5회 (끝 번호 -1)
range(1, 10, 2)	1, 3, 5, 7, 9	5회 (2씩 증가)

```
for i in range(1, 4): # i에 1, 2, 3이 차례로 들어갑니다.  
    print(f"{i}번째 반복 중...")
```

AI 모델을 학습시킬 때, 데이터를 한 번만 보여주지 않고 수십 번 반복해서 공부시킵니다.

이때 전체 데이터를 몇 번 반복해서 볼 것인지를 '**Epoch(에폭)**'이라고 하며, 이는 거대한 for 문으로 구현됩니다.

```
# 전체 데이터를 5번 반복해서 학습한다고 가정
epochs = 5

for epoch in range(1, epochs + 1):
    print(f" [Epoch {epoch}/{epochs}] 학습을 시작합니다...")
    # (여기서 실제로 복잡한 수학 연산과 학습이 일어납니다)
    print(f" {epoch}번째 학습 완료! 오차(Error)가 줄어들었습니다.")
    print("-" * 30)
print("모든 AI 모델 학습이 완료되었습니다!")
```

- 조건식이 참(True)인 동안 계속 반복합니다.
- 반복 횟수를 정확히 모를 때 유용합니다.
- **주의** : 조건이 영원히 True라면 프로그램이 멈추지 않는 **무한 루프(Infinite Loop)**에 빠집니다.

(강제 종료 : Ctrl + C)

```
count = 0
while count < 3:
    print(f"현재 카운트: {count}")
    count += 1    # 이 줄이 없으면 count가 계속 0이라 무한 루프 발생!
```

➤ 반복문의 흐름을 강제로 바꿀 때 사용합니다.

제어문	기능	비유
break	반복문을 즉시 탈출합니다.	게임 종료!
continue	아래 코드를 실행하지 않고 다음 반복으로 건너웁니다.	이번 판 무효, 다시 시작!

```
# 1부터 10까지 출력하되, 5가 되면 멈춤
for i in range(1, 11):
    if i == 5:
        break
    print(i, end=" ")    # 출력: 1 2 3 4
```

- 반복문 안에 반복문이 있는 형태입니다.
- 시계의 시침과 분침을 생각하면 쉽습니다. 바깥쪽이 한 번 돌 때, 안쪽은 전체를 돕니다.

```
for i in range(2, 10):    # 2단부터 9단까지
    print(f'=== {i}단 ===')
    for j in range(1, 10): # 곱해지는 수 1~9
        print(f'{i} x {j} = {i*j}')
```



06

CHAPTER

데이터 모뎀 (자료구조)

- List
- Tuple
- Dictionary
- Set

➤ 데이터가 많아지면 변수 하나하나에 담기 어렵습니다.

관련된 데이터를 묶어서 관리하는 파이썬의 4가지 보물 상자(리스트, 튜플, 딕셔너리, 집합)를 배웁니다.

구분	자료형	이름	특징	예시
기본 자료형	<code>int</code>	정수형	소수점이 없는 숫자	10, -5, 0
	<code>float</code>	실수형	소수점을 포함한 숫자	3.14, -0.1, 2.0
	<code>str</code>	문자열	텍스트 데이터 (따옴표 사용)	"Hello", 'Python'
	<code>bool</code>	논리형	참과 거짓을 판단	True, False
자료 구조형	<code>list</code>	리스트	여러 값을 순서대로 저장, 수정 가능	[1, 2, 3], ['a', 'b']
	<code>tuple</code>	튜플	여러 값을 순서대로 저장, 수정 불가	(1, 2, 3), ('x', 'y')
	<code>set</code>	집합	중복을 허용하지 않음, 순서 없음	{1, 2, 3}, {1, 1, 2} → {1, 2}
	<code>dict</code>	딕셔너리	키(Key)와 값(Value)의 쌍으로 구성	{"name": "Tom", "age": 20}
-	<code>None Type</code>	None	값이 존재하지 않음을 명시적으로 표현	None

➤ 정의

가장 많이 쓰이는 자료 구조입니다.

대괄호 []를 사용하며, 자유롭게 내용을 수정할 수 있습니다.

➤ 특징

- **순서 보장** : 데이터가 들어간 순서대로 저장되며, 인덱스(번호)로 접근할 수 있습니다.
- **가변(Mutable)** : 언제든지 데이터를 수정, 추가, 삭제할 수 있습니다.
- **다양한 자료형** : 숫자, 문자열, 불리언, 심지어 다른 리스트까지 섞어서 담을 수 있습니다.

➤ 용도

데이터의 목록을 관리하고 추가/삭제가 빈번할 때.

```
# 빈 리스트 만들기
```

```
empty_box = []
```

```
# 데이터 담기
```

```
fruits = ["사과", "바나나", "포도"]
```

```
print(fruits[0]) # 사과
```

```
# 데이터 수정하기
```

```
fruits[1] = "망고" # 바나나가 망고로 바뀜
```

분류	메서드	설명	예시 코드	코드 결과 (a 또는 반환값)
추가	append(x)	리스트의 끝에 항목 x를 하나 추가합니다.	a = [1, 2] a.append(3)	[1, 2, 3]
	extend(iterable)	리스트의 끝에 다른 리스트나 반복 가능한 객체의 모든 항목을 추가합니다.	a = [1, 2] a.extend([3, 4])	[1, 2, 3, 4]
	insert(i, x)	지정한 인덱스 i 위치에 항목 x를 삽입합니다.	a = [1, 3] a.insert(1, 2)	[1, 2, 3]
삭제	pop(i)	인덱스 i의 항목을 삭제하고 그 값을 반환합니다. (인덱스 생략 시 마지막 항목)	a = [1, 2, 3] val = a.pop()	a: [1, 2] val: 3
	remove(x)	리스트에서 첫 번째로 나오는 항목 x를 삭제합니다.	a = [1, 2, 3, 2] a.remove(2)	[1, 3, 2]
	clear()	리스트의 모든 항목을 삭제하여 빈 리스트로 만듭니다.	a = [1, 2, 3] a.clear()	[]
탐색/확인	index(x)	리스트에서 항목 x가 있는 첫 번째 인덱스를 반환합니다.	a = ['a', 'b', 'c'] idx = a.index('b')	1 (반환값)
	count(x)	리스트에 포함된 항목 x의 개수를 반환합니다.	a = [1, 2, 2, 3] cnt = a.count(2)	2 (반환값)
정렬/순서	sort()	리스트를 오름차순으로 원본 자체를 정렬합니다. (옵션: reverse=True 시 내림차순)	a = [3, 1, 2] a.sort()	[1, 2, 3]
	reverse()	리스트의 항목 순서를 거꾸로 뒤집습니다.(정렬 아님)	a = [1, 2, 3] a.reverse()	[3, 2, 1]
복사	copy()	리스트의 얇은 복사본(Shallow Copy)을 반환합니다.	a = [1, 2] b = a.copy()	b: [1, 2] (a와 b는 다른 객체)

```
# 리스트 생성
a = [0,1,2,3,4,5]

# 요소 추가(append)
a.append(6) # [0,1,2,3,4,5,6]
a.append([7,8]) # [0,1,2,3,4,5,6,[7,8]]

# 요소 접근
a[7][1] # 8

# 요소 삭제
a.pop() # [0,1,2,3,4,5,6]

# 요소 추가(extend)
a.extend([7,8]) # [0,1,2,3,4,5,6,7,8]

# 요소 변경
a[1] = "b" # [0,"b",2,3,4,5,6,7,8]

# 요소 복제하기
a*2 # [0,"b",2,3,4,5,6,7,8, 0,"b",2,3,4,5,6,7,8 ]
```

- 파이썬을 파이썬답게 쓰는 최고의 문법입니다.
- for 반복문을 한 줄로 압축하여 새로운 리스트를 만듭니다.

종류	정의	문법	사례
Type1	기본형 모든 데이터 변환	[표현식 for 항목 in 리스트]	<pre>nums = [1, 2, 3, 4, 5] # 모든 숫자를 10배로 만들기 result = [n * 10 for n in nums] print(result) # [10, 20, 30, 40, 50]</pre>
Type2	필터링형 특정 조건을 만족하는 데이터만 남깁니다. (if가 뒤에 옵니다)	[표현식 for 항목 in 리스트 if 조건]	<pre>nums = [1, 2, 3, 4, 5] # 짝수만 남기기 evens = [n for n in nums if n % 2 == 0] print(evens) # [2, 4]</pre>
Type3	선택형 조건에 따라 값을 다르게 채웁니다. (if~else가 앞에 옵니다)	[참값 if 조건 else 거짓값 for 항목 in 리스트]	<pre>scores = [80, 50, 90, 40] # 60점 이상이면 '합격', 아니면 '불합격' results = ["합격" if s >= 60 else "불합격" for s in scores] print(results) # ['합격', '불합격', '합격', '불합격']</pre>

➤ 정의

리스트와 비슷하지만, 한 번 만들면 **절대 내용을 바꿀 수 없는** 자료구조입니다. '읽기 전용 리스트'라고 이해하면 쉽습니다. 소괄호 ()를 사용합니다.

➤ 특징

- **순서 보장** : 데이터가 들어간 순서대로 저장되며, 인덱스(번호)로 접근할 수 있습니다.
- **수정불가(Immutable)** : 한 번 만들면 내용을 바꿀 수 없다는 점이 결정적으로 다릅니다.
- **다양한 자료형** : 숫자, 문자열, 불리언, 심지어 다른 리스트까지 섞어서 담을 수 있습니다.

➤ 용도

프로그램이 실행되는 동안 절대로 바뀌면 안 되는 중요한 값(예: 은행 계좌 설정, 위도/경도 좌표, 화면 해상도 등)을 실수로 수정하는 것을 방지하기 위해 씁니다.

```
menu = ("돈까스", "치즈까스")  
# menu[0] = "우동" <-- 에러 발생! (수정 불가)  
print(menu[0]) # 읽는 것은 가능
```

➤ 튜플은 수정이 불가능하므로 추가/삭제 메서드가 없습니다. 주로 조회용 메서드만 있습니다.

메서드	설명	사용 예시
<code>count</code>	특정 값이 몇 개 있는지 셉니다.	<code>menu.count("돈까스")</code>
<code>index</code>	특정 값의 위치(인덱스)를 찾습니다.	<code>menu.index("치즈까스")</code>

```
menu = ("돈까스", "치즈까스")
# menu[0] = "우동" <-- 에러 발생! (수정 불가)
print(menu[0]) # 읽는 것은 가능
```

➤ 정의

사전처럼 **단어(Key)**와 **뜻(Value)**이 짝을 이루는 자료구조입니다.

중괄호 {}와 콜론 :를 사용합니다 : { Key1 : Value1, Key2 : Value2 ... }

➤ 특징

- **순서 없음** : 리스트가 번호(0, 1...)로 데이터를 찾는다면, 딕셔너리는 이름(Key)으로 데이터를 찾습니다.
- **수정가능(mutable)** : 언제든지 데이터를 수정, 추가, 삭제할 수 있습니다.
- **다양한 자료형** : 숫자, 문자열, 불리언, 심지어 다른 리스트까지 섞어서 담을 수 있습니다.

➤ 용도

이름표를 붙여 데이터를 관리해야 할 때 (예: 회원 정보 - name: 'Gemini')

```
profile = { "이름": "홍길동", "나이": 20, "취미": ["독서", "코딩"] } # 리스트도 값으로 넣을 수 있음
print(profile["이름"])      # Key로 Value 찾기 '홍길동' 출력
profile["직업"] = "개발자"   # 새로운 키-값 쌍 추가
```

메서드	설명	사용 예시
keys	모든 키(Key)들의 목록을 가져옵니다.	<code>profile.keys()</code>
values	모든 값(Value)들의 목록을 가져옵니다.	<code>profile.values()</code>
items	(키, 값) 쌍을 튜플 형태로 모두 가져옵니다.	<code>profile.items()</code>
get	키로 값을 찾되, 없어도 에러 없이 None을 반환합니다.	<code>profile.get("키", "기본값")</code>
update	다른 딕셔너리 내용을 합치거나 수정합니다.	<code>profile.update(...)</code>

딕셔너리 생성

```
student = {"class" : [1,2,3,4], "name" : ["홍길동", "김철수", "이영희", "박영수"]}
```

```
student.keys() # dict_keys(['class', 'name'])
```

```
student.values() # dict_values([[1, 2, 3, 4], ['홍길동', '김철수', '이영희', '박영수']])
```

```
for key, value in student.items():
    print(f"{key} : {value}")
```

```
# class : [1, 2, 3, 4]
```

```
# name : ['홍길동', '김철수', '이영희', '박영수']
```

추가하기

```
student["score"] = [50,60,70,80]
```

데이터 분석 라이브러리인 **Pandas**의 핵심인 'DataFrame(표)'은 list, dictionary를 이용하여 만들어집니다.

```
Import pandas as pd
Student = {"class" : [1,2,3,4], "name" : ["홍길동", "김철수", "이영희", "박영수"], "score" :
[50,60,70,80]}
student = pd.DataFrame(student)
print(student)
```

	class	name	score
0	1	홍길동	50
1	2	김철수	60
2	3	이영희	70
3	4	박영수	80

```
student["score"].mean() # 65.0
```

➤ 정의

수학의 '집합'과 똑같습니다.
괄호 ()를 사용합니다.

➤ 특징

- 순서 없음 : 순서가 없습니다. 그래서 인덱싱이 불가능합니다.
- 중복 불가 : 중복된 데이터를 허용하지 않으며 또는 합집합/교집합 연산이 필요할 때 사용합니다.
- 수정가능(mutable) : 언제든지 데이터를 수정, 추가, 삭제할 수 있습니다.

➤ 용도

데이터에서 중복을 제거하고 싶을 때, 또는 합집합/교집합 연산이 필요할 때 사용합니다.

```
nums = [1, 1, 2, 2, 3, 3]
unique_nums = set(nums) # 집합으로 변환하여 중복 제거
print(unique_nums) # {1, 2, 3}
```

메서드	설명	사용 예시
add	값 1개를 추가합니다.	<code>s.add(4)</code>
update	여러 값을 한꺼번에 추가합니다.	<code>s.update([5, 6])</code>
remove	특정 값을 삭제합니다. (없으면 에러)	<code>s.remove(1)</code>
union	합집합을 구합니다. ()	<code>s1 s2</code>
intersection	교집합을 구합니다. (&)	<code>s1.intersection(s2)</code>
difference	차집합을 구합니다. (-)	<code>s1.difference(s2)</code>

- 파이썬의 자료형은 크게 '내용을 바꿀 수 있는 것'과 '없는 것'으로 나뉩니다.

종류	정의	대상	복사
가변(Mutable)	생성 후에도 내용을 바꿀 수 있습니다.	리스트, 딕셔너리, 집합	copy() 메서드 활용
불변(Immutable)	생성 후에는 내용을 바꿀 수 없습니다.	정수, 실수, 문자열, 튜플	

복사의 함정 : 참조 복사

리스트 같은 가변 객체를 = 등호로 다른 변수에 넣으면, 데이터 자체가 복사되는 것이 아니라 '주소표(참조)'만 복사됩니다. 즉, 두 변수가 하나의 데이터를 같이 바라보게 됩니다.

```
# [참조 복사]
a = [1, 2, 3]
b = a # 데이터 복사가 아니라, a가 가리키는 주소를 b도 가리키게 됨

b[0] = 100 # b를 수정했는데...
print(a)   # [100, 2, 3] -> 헉! a도 같이 바뀌어 버립니다.
```

➤ 진짜 복사하기: copy()

원본(a)을 보호하면서 복사본(b)만 수정하고 싶다면, copy() 메서드나 슬라이싱 [:]을 사용하여 **새로운 주소에 데이터를 복제**해야 합니다.

```
# [얕은 복사]
a = [1, 2, 3]
b = a.copy() # 또는 b = a[:]
b[0] = 100
print(a) # [1, 2, 3] -> 원본은 안전합니다.
print(b) # [100, 2, 3] -> 복사본만 바뀝니다.
```

07

CHAPTER

코드의 조립과 재사용 (함수)

- 함수 정의와 호출
- 변수의 범위

- 함수(Function)는 "입력을 넣으면 특정 기능을 수행하고 출력을 뱉는 기계"와 같습니다.
똑같은 코드를 여러 번 복사/붙여넣기 하지 않고, 이름만 불러서 재사용하기 위해 만듭니다.

➤ 함수의 기본 구조

파이썬에서는 def 키워드를 사용하여 함수를 정의합니다.

$f(x) = y$: 수학의 함수와 같이, 입력(x)을 넣으면 함수 내부의 로직을 거쳐 결과(y)를 내놓는 구조

- **def**: "이제부터 함수를 정의(define)하겠다"는 선언입니다.
- **함수 이름**: 함수를 호출할 때 사용할 이름입니다.
- **매개변수(Parameter)**: 함수 내부로 전달되는 입력값입니다.(생략 가능)
- **수행 문장**: 함수가 실행될 때 작동하는 코드입니다.
- **return**: 함수의 결과값을 밖으로 내보냅니다.(생략 가능)

형태	예시 코드	설명
입력 O, 출력 O	<code>def add(a, b): return a + b</code>	가장 일반적인 형태. 값을 받아 계산 후 반환.
입력 O, 출력 X	<code>def say_hi(name): print(f"Hi {name}")</code>	결과값을 반환하지 않고 특정 작업(출력 등)만 수행.
입력 X, 출력 O	<code>def get_pi(): return 3.14</code>	입력 없이 정해진 값을 반환.
입력 X, 출력 X	<code>def hello(): print("Hello!")</code>	호출하면 정해진 문장만 실행.

➤ 함수 정의와 호출

```
# 함수 정의 (기계 만들기)
def hello():
    print("안녕하세요!")
    print("반갑습니다.")
# 함수 호출 (기계 작동)
hello()
hello() # 여러 번 사용 가능
```

➤ 값을 받아 계산 후 반환

```
def add(a, b):          # a, b는 재료(매개변수)
    result = a + b
    return result       # 결과물을 밖으로 던져줌
total = add(3, 5)       # 3과 5를 넣어서 나온 결과(8)를 받음
print(total)
```

➤ 기본값 매개변수 : 값을 넣지 않았을 때 사용할 기본값을 지정할 수 있습니다.

```
def greet(name="친구"):
    print(f"{name}아, 안녕?")
greet()          # "친구아, 안녕?"
greet("철수")    # "철수아, 안녕?"
```

Q 문제 : 사용자로부터 원화(KRW) 금액과 현재 환율(1달러당 원화 가격)을 입력받아, 이를 달러(USD)로 환산하여 반환하는 함수 `convert_to_usd`를 작성하세요.

[조건]

- ① 함수 이름은 `convert_to_usd`로 합니다.
- ② 매개변수는 `krw_amount`(원화 금액)와 `exchange_rate`(환율) 두 개를 받습니다.
- ③ 계산된 달러 값을 `return`을 통해 반환해야 합니다.
- ④ 함수를 호출하여 1,000,000원을 환율 1,350원 기준으로 계산한 결과를 출력해 보세요.

Prompt

... [역할] 너는 파이썬 개발 전문가야

[상황] 원화(KRW)를 달러(USD)로 환산하는 함수를 작성하려고 해.

[지시] 함수 이름은 `convert_to_usd`로 지정할 것.

매개변수(parameter)는 `krw_amount`(원화 금액)와 `exchange_rate`(환율) 두 개를 받을 것.

함수 내부에서 원화를 환율로 나누어 달러 값을 계산하고, `return`을 사용하여 결과를 반환할 것.

[출력] 결과는 f-string을 사용하여 출력할 것.

출력 시 소수점 둘째 자리까지 표시되도록 f-string을 활용할 것. (예: 740.74 달러)

```
# [지시] 원화(KRW)를 달러(USD)로 환산하는 함수 정의
def convert_to_usd(krw_amount, exchange_rate):
    """ krw_amount: 원화 금액 (정수 또는 실수)
        exchange_rate: 현재 환율 (1달러당 원화 가격) """
    # 함수 내부에서 달러 값 계산
    usd_value = krw_amount / exchange_rate
    # [지시] return을 사용하여 결과 반환
    return usd_value

# 실제 사용 예시
krw = 1000000 # 100만 원
rate = 1350.5 # 예시 환율
# 함수 호출 및 결과 저장
result = convert_to_usd(krw, rate)

# [출력] f-string을 사용하여 소수점 둘째 자리까지 표시 (:2f 활용)
print(f"환산 결과: {result:.2f} 달러입니다.")
```

➤ 변수의 범위

- **지역 변수(Local)** : 함수 안에서 만든 변수. 함수가 끝나면 사라집니다. (밖에서 못 씀)
- **전역 변수(Global)** : 함수 **밖**에서 만든 변수. 어디서든 쓸 수 있습니다.
- **주의** : 함수 안에서 전역 변수를 수정하려면 `global` 키워드가 필요하지만, 남용하면 코드가 꼬일 수 있어 주의해야 합니다.

```
global_var = 100 # 전역 변수

def my_func():
    local_var = 20 # 지역 변수
    print(global_var) # 읽기 가능

my_func()
# print(local_var) # 에러! (함수 밖에서는 지역 변수가 안 보임)
```

➤ 이름 없는 함수 : lambda

보통 함수는 def로 이름을 짓고 만들지만, 한 번만 쓰고 버릴 간단한 함수는 람다(Lambda)로 만듭니다. 익명 함수라고도 부릅니다.

- 문법 : lambda 매개변수: 표현식
- 특징 : return 문 없이도 결과가 자동으로 반환됩니다.

```
# 일반 함수
def add(a, b):
    return a + b
# 람다 함수 (위와 같은 기능, 이름 없음)
add_lambda = lambda a, b: a + b
print(add_lambda(3, 5)) # 8
```

➤ 한꺼번에 처리하기 : map() 함수

리스트에 들어있는 모든 데이터에 똑같은 함수를 적용하고 싶을 때 map()을 사용합니다.

반복문(for)을 쓰지 않고도 한 줄로 처리할 수 있어 매우 효율적입니다.

- 문법 : map(함수, 리스트)
- 특징 : 결과물을 눈으로 확인하려면 list()로 감싸서 변환해야 합니다.

```
names = ["Kim Python", "Lee Java", "Park C++"]
# 공백 기준으로 자른 후([0]) 성만 가져오기
# def select_sur( x):
#     return x.split()[0] -> 입력된 문자열을 띄어쓰기로 자르고 첫 번째 단어 반환
# list(map(select_sur, names))
last_names = list(map(lambda x: x.split()[0], names))
print(last_names) # ['Kim', 'Lee', 'Park']
```

데이터 분석 라이브러리 Pandas에서는 수만 개의 데이터에 똑같은 연산을 적용할 때 `apply()`라는 기능을 쓰는 데, 이때 **람다 함수**가 필수적으로 사용됩니다.(DataFrame에서는 `map()` 대신 `apply()` 사용)

학생들의 이름과 점수가 담긴 데이터프레임이 있습니다. 점수(Score) 열을 기준으로 90점 이상은 '**A**', 80점 이상은 '**B**', 그 외에는 '**C**' 등급을 부여하는 등급(Grade) 열을 새로 추가하세요.

```
import pandas as pd

# 1. 샘플 데이터 생성
data = { 'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'], 'Score': [95, 82, 75, 89, 60] }
df = pd.DataFrame(data)

# 2. apply와 lambda를 사용하여 등급(Grade) 계산
# 로직: 90점 이상 'A', 아니면 (80점 이상 'B', 아니면 'C')
df['Grade'] = df['Score'].apply( lambda x: 'A' if x >= 90 else ('B' if x >= 80 else 'C') )

# 3. 결과 출력
print("--- 성적 결과 데이터프레임 ---")
print(df)
```

➤ 가변인자

파이썬의 가변인자(Variable-length Arguments)는 함수를 정의할 때 인자의 개수를 정해놓지 않고, 호출할 때마다 원하는 만큼 인자를 전달할 수 있게 해주는 아주 유용한 기능입니다.

➤ 위치 가변인자 (*args)

인자의 이름을 정하지 않고 순서대로(위치에 따라) 값을 전달할 때 사용합니다.

함수 내부에서 이 값들은 **튜플(Tuple)** 형태로 다뤄집니다.

- **표기** : 매개변수 앞에 별표(*) 하나를 붙입니다. (args는 관례적인 이름일 뿐, *numbers처럼 자유롭게 지어도 됩니다.)
- **특징** : 여러 개의 인자를 한꺼번에 묶어서 처리할 때 좋습니다.

```
def sum_all(*args):  
    # args는 들어온 값들을 (1, 2, 3) 처럼 튜플로 묶습니다.  
    return sum(args)  
  
print(sum_all(10, 20)) # 30  
print(sum_all(1, 2, 3, 4, 5)) # 15
```

➤ 키워드 가변인자 (**kwargs)

인자의 이름(Key)과 값(Value)을 한 쌍으로 전달할 때 사용합니다.

함수 내부에서 이 값들은 **딕셔너리(Dictionary)** 형태로 다뤄집니다.

- **표기** : 매개변수 앞에 별표 두 개(**)를 붙입니다. (Keyword Arguments의 약자입니다.)
- **특징** : 설정값이나 옵션처럼 데이터에 이름표를 붙여서 전달해야 할 때 유용합니다.

```
def print_user_info(**kwargs):  
    # kwargs는 {'name': 'Gemini', 'age': 20} 처럼 딕셔너리로 묶입니다.  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_user_info(name="Gemini", version="3.0")    # 출력: # name: Gemini # version: 3.0
```

08

CHAPTER

기능을 확장하는 방법 (모듈과 패키지)

- 모듈(Module)과 import
- 외부 패키지 설치: pip

➤ 라이브러리

- 자주 사용하는 프로그램을 **모듈과 패키지로 묶어서 구성**
- 파이썬에 기본적으로 설치되는 라이브러리를 **표준 라이브러리**라 함
(사용자가 별도의 설치 필요 없음)
※ 라이브러리, 패키지를 혼용해서 표현하기도 함

➤ 패키지

- 특정 기능과 관련된 **여러 모듈을 하나의 폴더에 넣은 것을 패키지**라 함
- 모듈이 많아지면 폴더 단위로 관리하게 되는데, 이를 **패키지**라고 부릅니다. 점(.)을 이용해 계층적으로 모듈을 불러옵니다.
- 패키지는 모듈별로 파일들을 구분하여, 하위 폴더에 저장
- 파이썬 작업환경에 필요한 패키지 설치 필요

➤ 모듈

- 특정 기능들(함수, 변수, 클래스 등)이 구현되어 있는 파이썬 파일(.py)
- 파이썬 코딩시 필요한 **모듈을 import 하여 사용**

➤ 모듈 (Module)이란?

모듈은 파이썬 코드를 담고 있는 **.py 파일** 그 자체입니다.

변수, 함수, 클래스 등을 모아놓은 파일이며, 다른 프로그램에서 이 파일의 기능을 가져다 쓸 수 있습니다.

- **표준 모듈** : 파이썬을 설치할 때 기본으로 제공되는 모듈 (예 : math, datetime, random)
- **사용자 정의 모듈** : 사용자가 직접 만든 .py 파일
- **외부 라이브러리** : 다른 사람들이 만들어 배포한 모듈 (예 : pandas, numpy) - pip install로 설치 필요

➤ import 사용법 : 다른 모듈의 기능을 가져올 때는 import 키워드를 사용합니다.

① **기본 사용법** - 모듈 전체를 가져옵니다. 함수를 쓸 때는 **모듈명.함수명()** 형식을 사용합니다.

```
import math # 수학 관련 모듈 가져오기
print(math.pi) # 3.141592...
print(math.sqrt(16)) # 4.0 (루트 16)
```

② 특정 기능만 가져오기(from ... import)

모듈 안에서 필요한 함수나 클래스만 콧 집어서 가져옵니다. 이때는 모듈명을 생략하고 바로 쓸 수 있습니다.

```
from math import sqrt, pi
print(sqrt(16)) # 4.0
```

➤ 모듈의 동작 원리

파이썬이 모듈을 찾는 순서와 구조를 이해하면 더 깊이 있는 코딩이 가능합니다.

1. **sys.modules:** 이미 로드된 모듈인지 먼저 확인합니다.
2. **Built-in Modules:** 파이썬 기본 내장 모듈인지 확인합니다.
3. **sys.path:** 리스트 형태의 경로들을 순서대로 뒤져서 해당 .py 파일을 찾습니다.

➤ 패키지

모듈이 많아지면 폴더 단위로 관리하게 되는데, 이를 패키지라고 부릅니다. 점(.)을 이용해 계층적으로 모듈을 불러옵니다.

```
# 'sound' 패키지의 'effects' 폴더 안의 'echo' 모듈 호출  
import sound.effects.echo
```

➤ 파이썬을 설치하면 기본적으로 따라오는 유용한 모듈

모듈	설명	주요 함수
random	난수(무작위 수) 생성	random.randint(1, 10) (1~10 랜덤 정수)
datetime	날짜와 시간 다루기	datetime.date.today() (오늘 날짜)
time	시간 지연(잠시 멈춤)	time.sleep(1) (1초 멈춤)
os	운영체제 제어(파일 경로 등)	os.getcwd() (현재 폴더 위치)

➤ 오늘부터 올해 크리스마스(12월 25일)까지 남은 날짜 계산하기

```
import datetime

today = datetime.date.today() # 오늘 날짜
christmas = datetime.date(today.year, 12, 25) # 올해 크리스마스 (올해 연도, 12 월, 25 일)

d_day = christmas - today # 남은 날짜 계산
print(f"오늘: {today}")
print(f"크리스마스까지 {d_day.days}일 남았습니다! 🎅 ")
```

➤ PyPI (Python Package Index)

대부분의 외부 패키지는 **PyPI**라는 거대한 저장소에 보관되어 있습니다.

전 세계 누구나 자신이 만든 패키지를 올릴 수 있고, 누구나 내려받아 사용할 수 있습니다.

- 공식 홈페이지: pypi.org
- 규모: 수십만 개의 패키지가 존재하며 매일 업데이트됩니다.

➤ 패키지 관리자 : pip

외부 패키지를 내 컴퓨터에 설치하고 관리하려면 pip (Python Package Installer)라는 도구를 사용합니다.

터미널이나 CMD창에서 명령어를 입력합니다.

명령어	내용
<code>pip install --upgrade pip</code>	pip 명령어 업그레이드
<code>pip list</code>	현재 환경에 설치된 패키지들을 보여줌
<code>pip install 패키지명</code>	외부 패키지 설치, 일반적으로 압축파일 형태로 특정 사이트에서 제공
<code>pip install 패키지명==버전번호</code>	특정 버전의 패키지 설치 ex) <code>pip install pandas==3.0.0</code>
<code>pip install --upgrade 패키지명</code>	특정 패키지 업그레이드
<code>pip uninstall 패키지명</code>	특정 패키지 삭제
<code>pip show 패키지명</code>	특정 패키지 정보확인

➤ 외부 패키지

패키지명	주요기능
pandas	테이블 형태의 데이터를 다루는 데이터프레임(DataFrame) 자료 핸들링
numpy	수치 해석, 특히 선형 대수(linear algebra)의 다차원 배열, 벡터 연산
matplotlib	각종 그래프나 차트 등을 그리는 시각화 기능
Scipy	고급 수학 함수, 수치 미적분, 미분 방정식 계산, 최적화, 신호 처리
Seaborn	matplotlib 패키지에서 지원하지 않는 고급 통계 차트 등 시각화 기능
StatsModels	통계 및 회귀 분석이나 시계열 분석
Scikit-Learn	대부의 머신러닝 모델 제공, 파이썬으로 머신러닝을 공부하는데 최적의 패키지임
TensorFlow	신경망 모형 등 딥러닝 모델 제공, 심볼릭 연산과 그래프 연산 모형, GPU를 사용한 연산 등
BeautifulSoup / Selenium	인터넷 페이지의 정보를 수집할 때 사용합니다.
Langchain/Langgraph	LLM을 활용한 애플리케이션을 더 쉽게 개발할 수 있도록 돕는 라이브러리 모음집

* pandas 활용 데이터 분석

pandas을 읽어와서 pd로 이름 붙여서 사용하기

```
import pandas as pd
```

```
student = {'no' : ['1번', '2번', '3번'], 'name' : ['김철수', '박민호', '황기수'], 'math' : [80,85,90],  
           "eng" : [50,70,80]}
```

dataframe 객체 생성하기

```
df = pd.DataFrame(student)
```

math, eng 합 계산하기

```
df["math"]+df["eng"]
```

math, eng 각 평균 계산하기

```
print(df["math"].mean())
```

```
print(df["eng"].mean())
```

math 80이상 학생 찾기

```
df[df["math"] > 80 ]
```

11

CHAPTER

실전! 미니 프로젝트 (GPT 활용)

➤ 지금까지 배운 변수, 제어문, 자료구조, 함수, 클래스,
파일 입출력을 총동원하여 프로그램을 만들어 봅시다.

Prompt



[역할] 너는 파이썬을 교육하는 선생님이야.

[상황] python 문법을 배우고 복습용으로 미니 프로젝트를 실습하려고 해

[지시] 변수, 연산, 제어문, 반복문, 자료구조, 함수 등을 사용하여 개인 가계부 관리 프로그램을 작성해줘. 프로그램은 아래와 같은 기능을 포함해줘.

1. 콘솔에서 동작하는 아주 작은 가계부를 만든다.
2. 사용자는 메뉴를 보고 숫자를 입력한다.
3. 수입/지출을 기록한다.
4. 기록 목록을 조회할 수 있다.
5. 총합/잔액을 보여 준다.

```
# 가계부 데이터 저장 (리스트 + 딕셔너리)
ledger = []
# 수입/지출 추가 함수
def add_record():
    print("\n[수입/지출 입력]")
    type_choice = input("1. 수입 / 2. 지출 선택: ")
    if type_choice == "1":
        record_type = "수입"
    elif type_choice == "2":
        record_type = "지출"
    else:
        print("잘못된 입력입니다.")
        return
    amount = int(input("금액 입력: "))
    description = input("내용 입력: ")

    # 딕셔너리로 저장
    record = { "type": record_type, "amount": amount, "description": description }
    ledger.append(record)
    print("기록이 저장되었습니다!")
```

```
# 전체 기록 조회
def show_records():
    print("\n[전체 기록]")
    if not ledger:
        print("기록이 없습니다.")
        return
    for i, record in enumerate(ledger, start=1):
        print(f"{i}. [{record['type']}] {record['amount']}원 - {record['description']}")

# 총합 및 잔액 계산
def show_summary():
    income = 0
    expense = 0
    for record in ledger:
        if record["type"] == "수입":
            income += record["amount"]
        else:
            expense += record["amount"]
    balance = income - expense
    print("\n[요약]")
    print(f"총 수입: {income}원")
    print(f"총 지출: {expense}원")
    print(f"잔액: {balance}원")
```

```
# 메인 실행 함수
def main():
    while True:
        print("\n===== 가계부 프로그램 =====")
        print("1. 수입/지출 입력")
        print("2. 기록 조회")
        print("3. 총합/잔액 보기")
        print("4. 종료")
        choice = input("메뉴 선택: ")
        if choice == "1":
            add_record()
        elif choice == "2":
            show_records()
        elif choice == "3":
            show_summary()
        elif choice == "4":
            print("프로그램 종료")
            break
        else:
            print("올바른 번호를 입력하세요.")# 프로그램 시작

main()
```

Prompt



[역할] 너는 데이터 분석 전문가야.

[상황] 데이터 분석을 통해서 숨겨진 패턴을 찾기 위해서 탐색적 분석을 하려고해

[지시] 아래 분석대로 분석해줘

1. data 폴더에서 co2_자동차배출량.csv 파일 불러오기
 - 한글이 포함되어 있을 경우 cp949를 사용해서 데이터를 불러와줘.
2. 데이터의 앞쪽 5개와 뒤쪽 5개 확인하기
3. 전체 데이터의 크기를 행과 열로 확인하기
4. 데이터의 컬럼별 특성 확인하기
5. 변수별 타입에 따라 기술통계량 확인하기
7. 결과변수 CO2배출량 분포 확인하기

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as 눈
# 한글 깨짐 방지 (윈도우 기준)
plt.rcParams['font.family'] = 'Malgun Gothic'

# 1. 데이터 불러오기 (cp949)
file_path = r"D:\작업방\data\dataiku_이러닝_L1(E직군)\co2_자동차배출량.csv"
df = pd.read_csv(file_path, encoding='cp949')

# 2. 데이터 앞/뒤 5개 확인
print("=== 상위 5개 ===")
print(df.head())
print("\n=== 하위 5개 ===")
print(df.tail())
# 3. 데이터 크기 확인
print("\n=== 데이터 크기 ===")
print(f"행: {df.shape[0]}, 열: {df.shape[1]}")

# 4. 컬럼별 특성 확인
print("\n=== 컬럼 정보 ===")
df.info()
print("\n=== 결측값 확인 ===")
print(df.isnull().sum())
```

```
# 5. 변수 타입별 기술통계량
print("\n=== 수치형 변수 통계 ===")
print(df.describe())print("\n=== 범주형 변수 통계 ===")
print(df.describe(include='object'))
```

```
# 7. CO2_배출량 분포 확인
plt.figure()
sns.histplot(df['CO2배출량'], kde=True)
plt.title("CO2 배출량 분포")
plt.xlabel("CO2 배출량")
plt.ylabel("빈도")
plt.show()
```

Prompt

... 구글에서 '포스코' 이미지를 icrawler패키지를 이용해서 크롤링하고
large 사이즈 이미지 10개 다운로드 받으려고 해.
파이썬 코드를 생성해줘 확인하기

```
# pip install icrawler

import os
from icrawler.builtin import GoogleImageCrawler

# 저장 폴더 생성
output_dir = "./posco_images"
os.makedirs(output_dir, exist_ok=True)

# 크롤러 생성
google_crawler = GoogleImageCrawler(
    storage={"root_dir": output_dir}
)

# large 사이즈 필터 적용하여 10장 다운로드
google_crawler.crawl(
    keyword="포스코",
    filters={"size": "large"},
    max_num=10
)

print("완료: ./posco_images 폴더에 large 이미지 10장이 저장되었습니다.")
```

Prompt



파이썬으로 네이버 뉴스에서 'AI 트렌드' 키워드로 검색된 최신 뉴스 제목 20개와 해당 기사 링크를 수집해줘. 수집한 결과는 '오늘의_뉴스.xlsx'라는 엑셀 파일로 저장하는 코드를 짜줘. pandas와 BeautifulSoup 라이브러리를 사용해줘

뉴스 수집 순서

1. 네이버 뉴스 검색 결과 페이지 불러오기
2. 뉴스 제목과 링크 데이터 추출하기
3. 수집된 데이터 개수 및 샘플 확인하기
4. 수집한 데이터를 표(DataFrame) 형태로 변환하기
5. 엑셀 파일로 저장 및 결과 확인하기

Prompt



[역할] 너는 파이썬 GUI 자동화 전문가야.

[상황] pyautogui 패키지를 사용하여 윈도우 그림판(mspaint)에 그림을 그리는 스크립트를 작성해줘.

[지시] 윈도우에서 mspaint 그림판을 실행해서 아래 내용을 참조해서 나선 모양의 그림을 그려줘.

각 단계 사이에 시스템 안정화를 위해 충분한 sleep 시간을 추가해줘

1. 그림판을 실행하기
2. 그림판을 최대화하기
3. 사용자가 색을 선택하도록 해주고, 선택되면 다음 단계 실시하기
4. 시작 위치를 사용자가 선택하도록 하고 선택하면 다음 단계 실시하기
5. 초기 길이를 200으로 설정하고, 사각형 나선을 그려줘
6. 이동 순서는 우측 -> 아래 -> 좌측 -> 위 순서이며, 한 번 이동할 때마다 0.5씩 감소시켜서 안쪽으로 말려 들어가는 모양을 만들어줘
7. 길이가 0이면 정지해
8. 마우스 이동은 drag 함수를 사용하고 duration은 0.5초로 설정해.
9. 그림을 다 그린 후 그림을 지워줘

Prompt

... [역할] 너는 데이터 분석 전문가야.

[상황] Streamlit 패키지를 활용하여 데이터 분석 대시보드 웹을 만들려고 해.

아래 내용의 순서대로 순차적으로 진행하는 파이썬 코드를 작성해서 보여주고, 웹 실행방법을 알려줘.

[지시] 그래프를 그릴때 아래 내용을 참조해서 파이썬 코드를 생성하고자 해.

- seaborn 그래프를 사용하고 한글이 깨지지 않도록 코드를 작성해줘
- Malgun Gothic(맑은고딕) 폰트를 반드시 사용하고,
- matplotlib rcParams 설정뿐 아니라 `seaborn.set_theme(rc=...)`까지 포함해서 폰트를 고정해줘
- seaborn 그래프를 그리기 전에 폰트 설정이 완료되도록 코드 순서를 보장해줘

데이터 불러와서 확인하기

1. 파일 업로드 : 여러개의 데이터셋을 불러오기

- 변수명에 한글이 포함되어 있으므로 `cp949`, `utf-8` 등을 사용해서 한글 불러오기 확인하기
- 데이터 정보(변수 명, 변수 타입, 데이터 전체 크기, 기술통계량) 확인
- 그래프를 그릴때 한글 폰트는 맑은고딕으로 설정하고, 표에 음수가 나오면 에러가 발생하지 않도록 처리해줘

Prompt

2. 대시보드 구성은 4등분해서 표현하고, 기술통계량, 히스토그램과 박스플롯, 산점도, 상관분석으로 구성해줘
- 1번 화면 : 기술통계량을 소수 2째자리까지 표현
 - 2번 화면 : 변수 선택시 히스토그램과 박스플롯 그리기, plotly사용해서 구현
 - 3번 화면 : 연속형 변수를 지정하여 산점도를 그리기 , plotly사용해서 구현
예) x축에 변수1, y축에 변수2 지정하면 산점도 그리기
 - 4번 화면 : 상관분석 결과 표현
예) 연속형 종속변수와 연속형 독립변수간 상관분석 진행하기 : 상관계수 히트맵으로 표현하고, 계수값을 추가하기
히트맵에 상관계수 추가할 경우 값이 히트맵 밖으로 벗어나지 않도록 폰트크기 조정하기조정하기

첨부

효율적인 설계도 (클래스와 객체)

- 객체 생성하기
- 클래스 변수와 인스턴스 변수
- 클래스 상속

➤ 지금까지는 변수와 함수를 따로 관리했습니다.

하지만 게임 캐릭터나 AI 모델처럼 "속성(변수)"과 "행동(함수)"을 하나로 묶어 관리해야 할 때가 많습니다.
이것이 바로 **객체지향 프로그래밍(OOP)**입니다.

- 클래스 (Class) : 붕어빵 틀 (설계도)
- 객체/인스턴스 (Object/Instance) : 틀로 찍어낸 붕어빵 (실체)

```
class FishBread:
    # 생성자: 붕어빵이 태어날 때 실행됨
    def __init__(self, flavor, price):
        self.flavor = flavor # 맛 (속성)
        self.price = price  # 가격 (속성)

# 메서드: 붕어빵의 행동
    def show_info(self):
        # 여기서 self는 호출한 녀석(bread1 또는 bread2)이 됩니다.
        print(f"{self.flavor} 붕어빵은 {self.price}원입니다.")

# 객체 생성 (틀로 빵 찍어내기)
bread1 = FishBread("팥", 1000)      # self가 bread1이 되어 '팥'을 저장
bread2 = FishBread("슈크림", 1200)  # self가 bread2가 되어 '슈크림'을 저장
bread1.show_info() # bread1의 정보 출력
bread2.show_info() # bread2의 정보 출력
```

- 클래스 안에는 변수와 함수(메서드)가 들어갑니다. 특히 `__init__`은 객체가 처음 만들어질 때 속성을 설정(초기화)하는 특수한 함수입니다.

1 self가 도대체 뭔가요?

`self`는 "나 자신"을 가리키는 말입니다. 붕어빵 틀(클래스)에서 붕어빵(객체)이 막 나왔을 때, 이 붕어빵이 "나는 팔 맛이야"라고 기억하려면 자기 자신(`self`)에게 이름표를 붙여줘야 합니다.

- `self.flavor = "팔"` → 나의 맛은 팔이다.
- `self.price = 1000` → 나의 가격은 1000원이다.

2 객체 생성 과정의 비밀

`bread1 = FishBread("팔", 1000)` 코드가 실행될 때, 컴퓨터 내부에서는 다음과 같은 일이 일어납니다.

- 1) 빈 붕어빵 탄생 : 파이썬이 메모리에 아무 내용 없는 빈 객체를 하나 만듭니다.
- 2) `self`로 전달 : 이 빈 객체를 `__init__` 함수의 첫 번째 파라미터인 `self`에게 쏙 넣어줍니다.
(즉, 지금부터 `self`는 방금 만든 그 빈 붕어빵입니다.)
- 3) 속성 부여 : `__init__` 함수 안의 `self.flavor = flavor` 코드가 실행되면서, 빈 붕어빵(`self`)에 '팔'이라는 내용물이 채워집니다.
- 4) 변수에 저장 : 내용물이 다 채워진 완성된 붕어빵이 `bread1`이라는 변수에 담깁니다.

3 메서드에서 self가 변하는 마법

객체를 만들고 나서 `bread1.show_info()`를 실행하면 무슨 일이 일어날까요?

- 1) 우리는 `show_info()` 괄호 안에 아무것도 안 넣었지만, 파이썬이 알아서 **주인공**을 넣어줍니다.
- 2) `bread1.show_info()` ➡ 실제로는 `FishBread.show_info(bread1)`처럼 동작합니다.
- 3) 함수 정의의 `def show_info(self):`에서 `self` 자리에 `bread1`이 들어가는 것이죠.
- 4) 그래서 `print(self.flavor)`는 `print(bread1.flavor)`가 되어 "팥"이 출력되는 것입니다.

➤ 클래스를 만들 때 변수를 어디에 적느냐에 따라 성격이 완전히 달라집니다. 이 차이를 아는 것이 중요합니다.

- 인스턴스 변수 (self.변수) : 객체(붕어빵)마다 각자 따로 가지는 값 (예: 내 배속의 팔 앙금)
- 클래스 변수 (클래스.변수) : 모든 객체가 다 같이 공유하는 값 (예: 붕어빵 가게 이름)

```
class Monster:
    count = 0  # [클래스 변수] 모든 몬스터가 공유 (총 몬스터 수)

    def __init__(self, name):
        self.name = name  # [인스턴스 변수] 몬스터마다 다름 (내 이름)
        Monster.count += 1  # 몬스터가 태어날 때마다 전체 카운트 증가

m1 = Monster("슬라임")
print(f"현재 몬스터 수: {Monster.count}마리")  # 1마리

m2 = Monster("골렘")
print(f"현재 몬스터 수: {Monster.count}마리")  # 2마리 (m1에게도 영향을 줌)
```

- 이미 만들어진 클래스의 기능을 물려받아 새로운 기능을 추가하는 것입니다.
(예: '붕어빵' 클래스를 상속받아 '잉어빵' 만들기)

```
# FishBread의 기능을 물려받음
class PremiumFishBread(FishBread):
    def discount(self):
        self.price -= 100 # 100원 할인 기능 추가

p_bread = PremiumFishBread("황금", 2000)
p_bread.discount()
p_bread.show_info() # 황금 붕어빵은 1900원입니다.
```

[연습문제] 자동차 공장

Q 문제 : Car 클래스를 만드세요.

- [조건]
- ① 속성 : model(모델명), speed(속도, 초기값 0)
 - ② 메서드 accelerate(value): 속도가 value만큼 증가
 - ③ 메서드 brake(value): 속도가 value만큼 감소 (0 밑으로는 안 내려감)
 - ④ 차를 한 대 뽑아서 달리고 멈춰보세요.

```
class Car:
    def __init__(self, model):
        self.model = model
        self.speed = 0

    def accelerate(self, value):
        self.speed += value
        print(f"{self.model} 가속! 현재 속도: {self.speed}km/h")

    def brake(self, value):
        self.speed -= value
        if self.speed < 0:
            self.speed = 0
        print(f"{self.model} 감속! 현재 속도: {self.speed}km/h")

my_car = Car("테슬라")
my_car.accelerate(50)
my_car.brake(20)
```

첨 부

파일 입출력과 예외 처리

- 파일 생성하고 읽고 쓰기
- 안정적인 프로그램 만들기 (예외 처리)

- 파이썬에서는 `open()` 함수 하나로 파일을 만들거나 읽을 수 있습니다.

모드	설명	주의사항
w	쓰기 (Write)	파일이 없으면 새로 만들고, 있으면 기존 내용을 다 지우고 씁니다.
r	읽기 (Read)	파일이 없으면 에러가 발생합니다.
a	추가 (Append)	기존 내용 뒤에 새로운 내용을 덧붙입니다.

파일 쓰기 (w)

```
file = open("memo.txt", "w", encoding="utf-8") # 한글 깨짐 방지
file.write("파이썬 파일 입출력 테스트\n")
file.close() # 꼭 닫아줘야 합니다!
```

파일 읽기 (r)

```
file = open("memo.txt", "r", encoding="utf-8")
content = file.read()
print(content)
file.close()
```

TIP! 왜 `encoding="utf-8"`을 쓰나요?

이 옵션 없이 저장하면 다른 컴퓨터에서 열었을 때 '의힘...' 처럼 글자가 깨질 수 있습니다. 한글을 다룬다면 무조건 붙이는 습관을 들이세요!

- 파일을 열고 `close()` 하는 것을 깜빡하면 컴퓨터 메모리에 문제가 생길 수 있습니다.
with 문을 쓰면 블록이 끝날 때 **자동으로 파일을 닫아줍니다.** (권장 방법)

with를 쓰면 close()가 필요 없음

```
with open("memo.txt", "a", encoding="utf-8") as f:  
    f.write("이 내용은 자동으로 저장되고 닫힙니다.\n")
```

➤ 파일 내용을 읽어오는 3가지 방법

데이터 분석에서는 파일 전체를 한 번에 읽기보다 한 줄씩 읽어야 할 때가 많습니다.

메서드	설명	반환 형태
<code>read()</code>	파일 내용 전체 를 하나의 문자열로 읽습니다.	문자열 (str)
<code>readline()</code>	한 줄만 읽습니다. 반복문과 함께 씁니다.	문자열 (str)
<code>readlines()</code>	전체를 읽어서 줄 단위로 리스트 에 담습니다.	리스트 (list)

- 프로그램 실행 중 에러는 피할 수 없습니다.
핵심은 에러가 났을 때 프로그램이 비정상 종료되는 것을 막고, 하는 것입니다.

- **try ~ except 구문**

에러가 발생할 가능성이 있는 코드를 try에, 에러 발생 시 처리할 코드를 except에 작성합니다.

```
try:
    # 에러가 날 수도 있는 코드
    result = 10 / 0
except ZeroDivisionError:
    # 에러가 났을 때 실행할 코드
    print("0으로 나눌 수 없습니다!")
```